



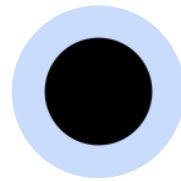
DATA SCIENCE CASE STUDY

Loan Offer Propensity Model

Chuene Mosomane

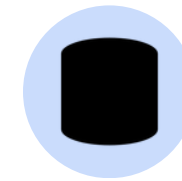


What we'll cover



Business Problem

The bank wants to offer their customers a product - loan. It wants to identify which customers to approach, and if there is any bias in the data.



Data & EDA findings

We have three tables:

- customers
- loans
- transactions

What the three source actually looks like.



Propensity Modelling

- I develop a 2 levels-framework - customer and loan levels
- Then build a classification model to predict the probability of a which customers will take up a loan.
- The customers identified are stored in a CSV file



Feature engineering

I engineered financial features to check the financial-health KPIs behind the model, in plain terms:

- Net cash flow, balance volatility, low balance frequency
- Balance to income ratio, burn rate.



Model & fairness results

- I performed feature selection to rank discriminant features.
- Then train and evaluate a Logistic regression with 85/15 split.
- Lastly generate a fairness report: approval by age and gender



MLOps: pipeline, monitoring, next steps

- The solution is written in PySpark and runs in production.
- Monitoring for key metrics is done in Grafana dashboards
- Recommendations: This solution is used by the marketing team for promotional intervention.

Which customers should we approach with a loan offer?

Objective

Use exploratory analytics, visualization, and machine learning on customer, transaction, and loan data to identify clients we should proactively approach with a loan offer.

Why it matters

- Targeted offers convert better than blanket campaigns and cost less to run
- Getting this wrong at scale means either missed revenue or reputational/regulatory risk
- A model that can't explain *why* it flags someone, it is a liability in a regulated lending context

Three tables, one customer ID to join them

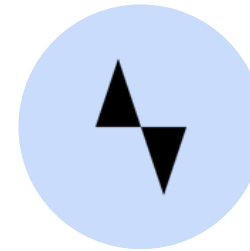
```
customers: 2500 rows  
loans: 3203 rows  
transactions: 91931 rows
```



Customers

- Gender
- Age
- Income

One row per client



Transactions

- Date
- Amount (+in / -out)
- Balance after transaction

Many rows per client, over time



Loans

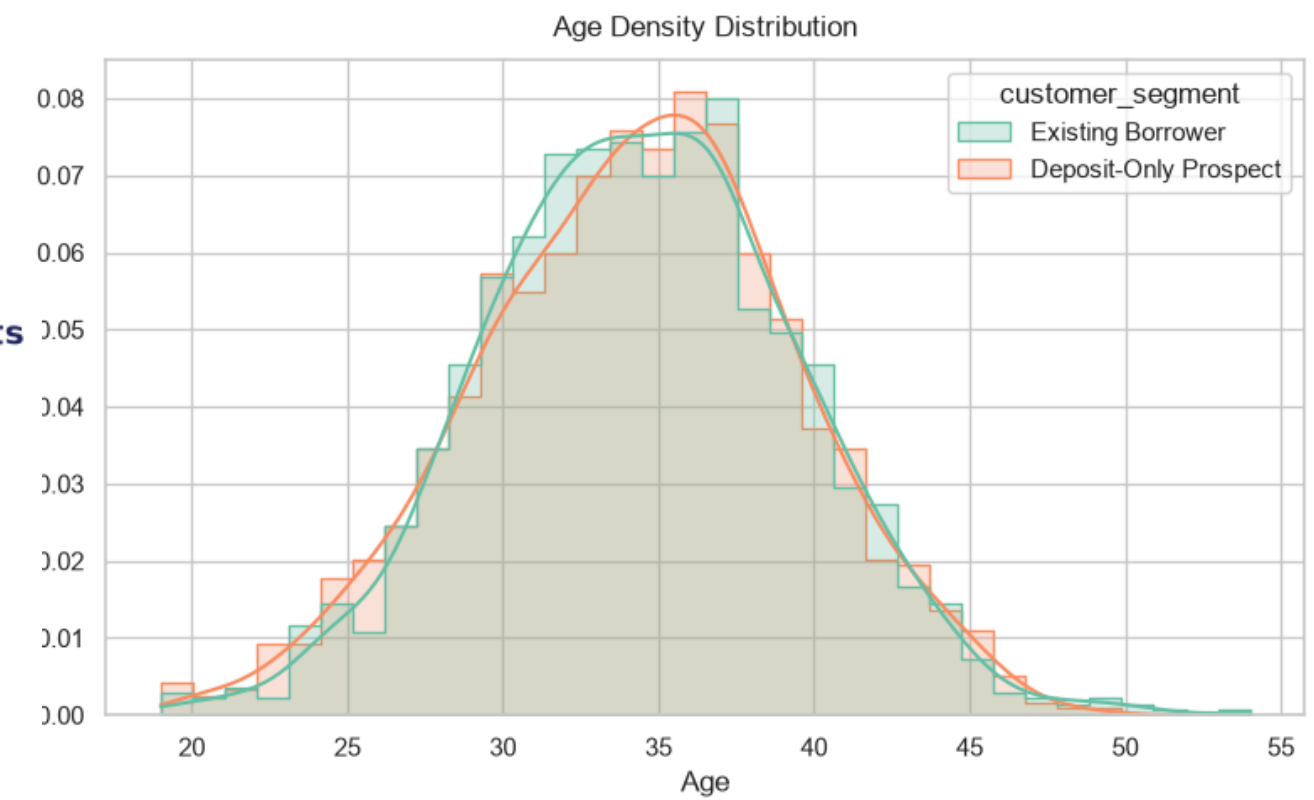
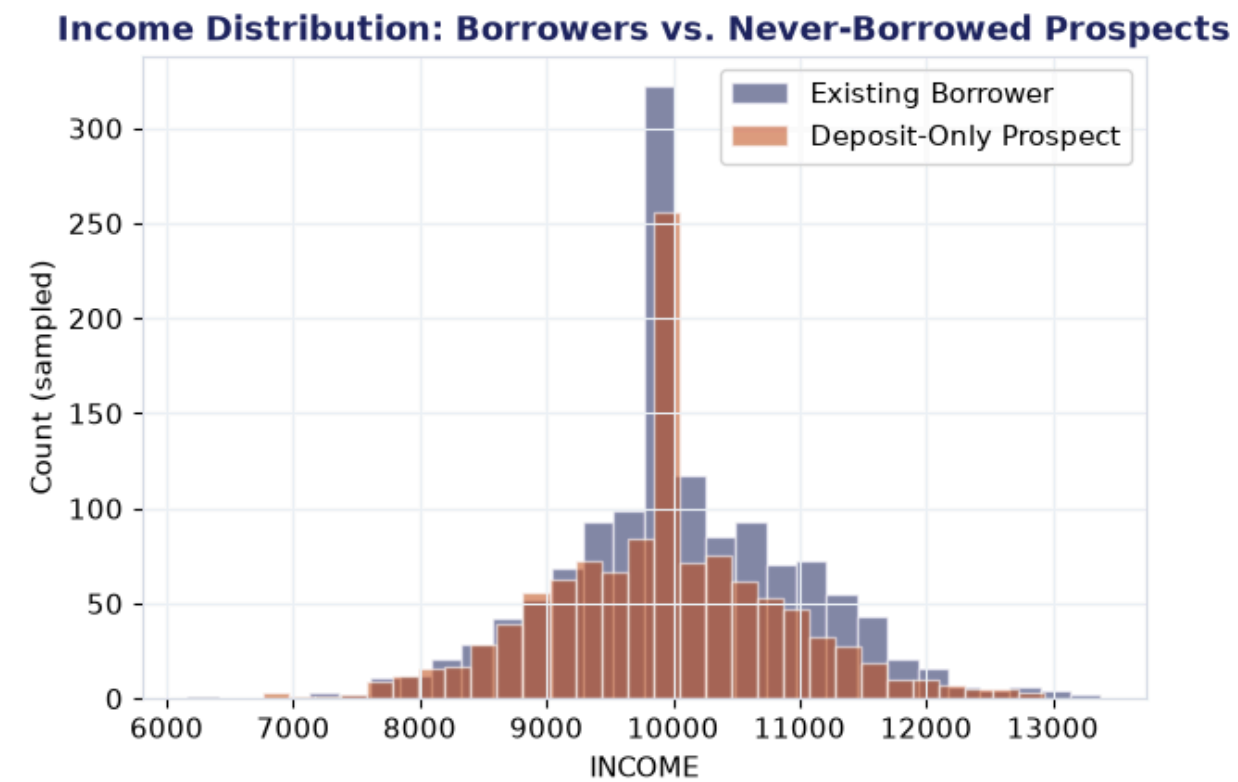
- Date applied
- Amount requested
- Outcome: TakeUp / Declined

One row per historical application

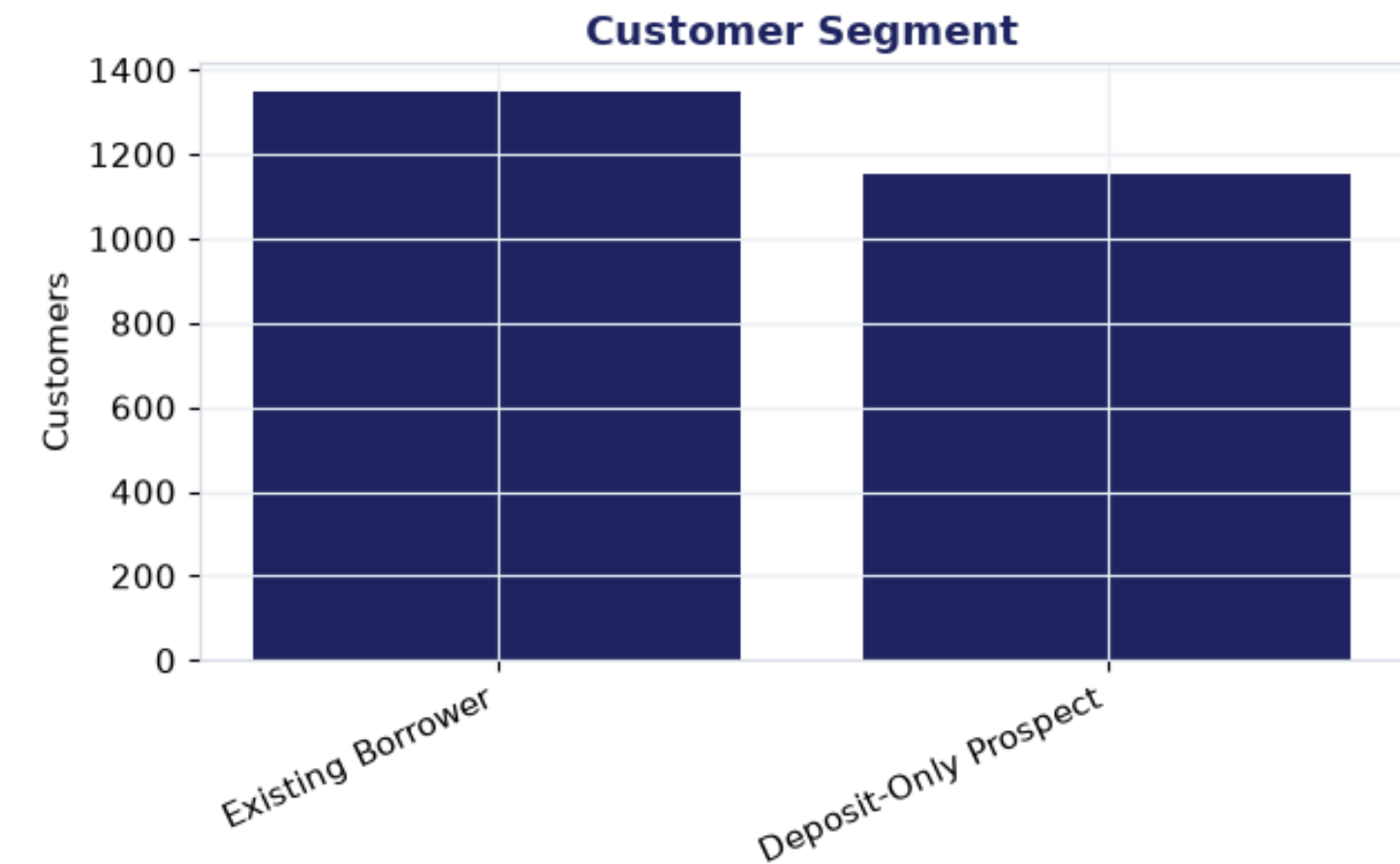


Joined on customer ID into two working tables: a customer-level view (for EDA and segmentation) and a loan-application-level view for modelling.

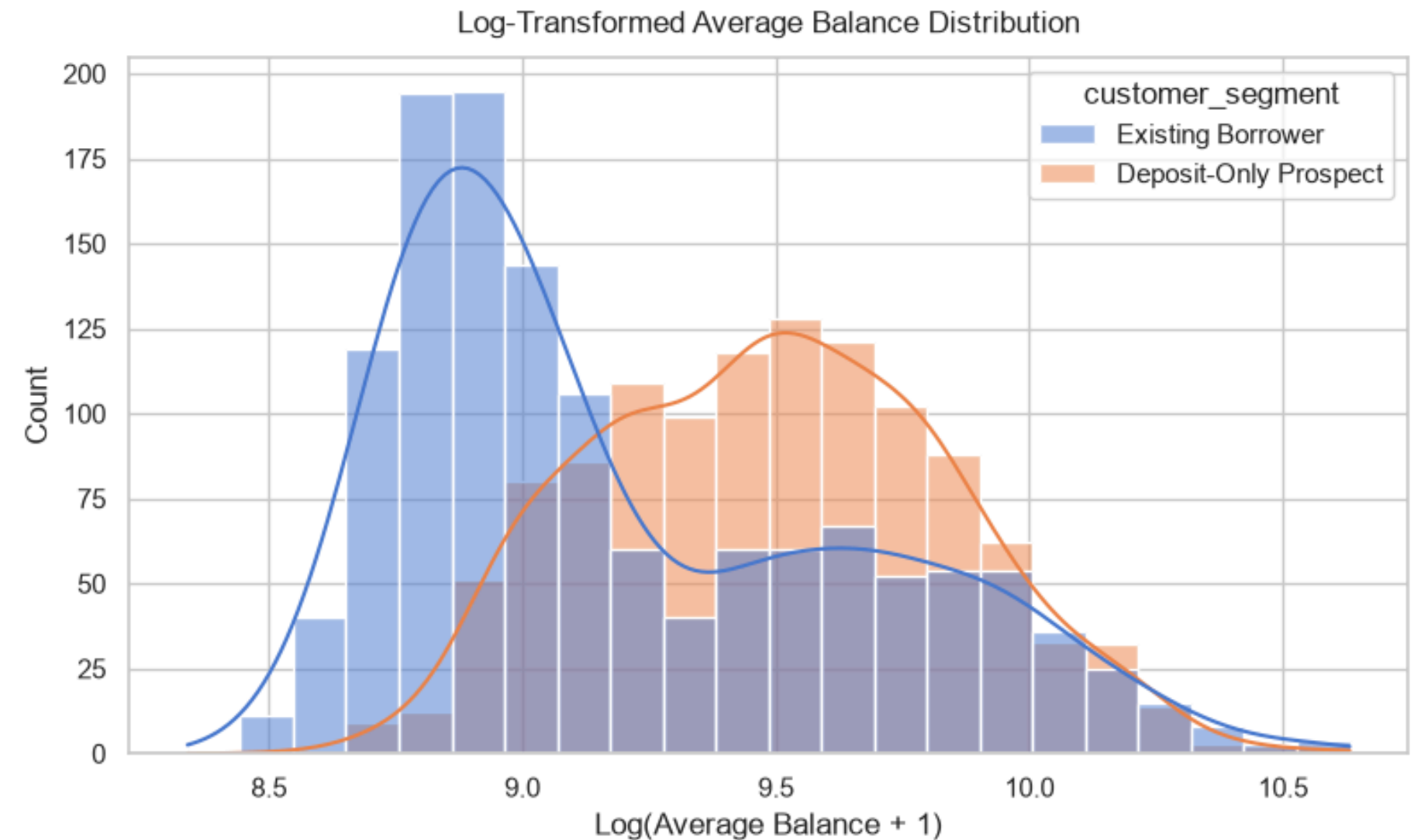
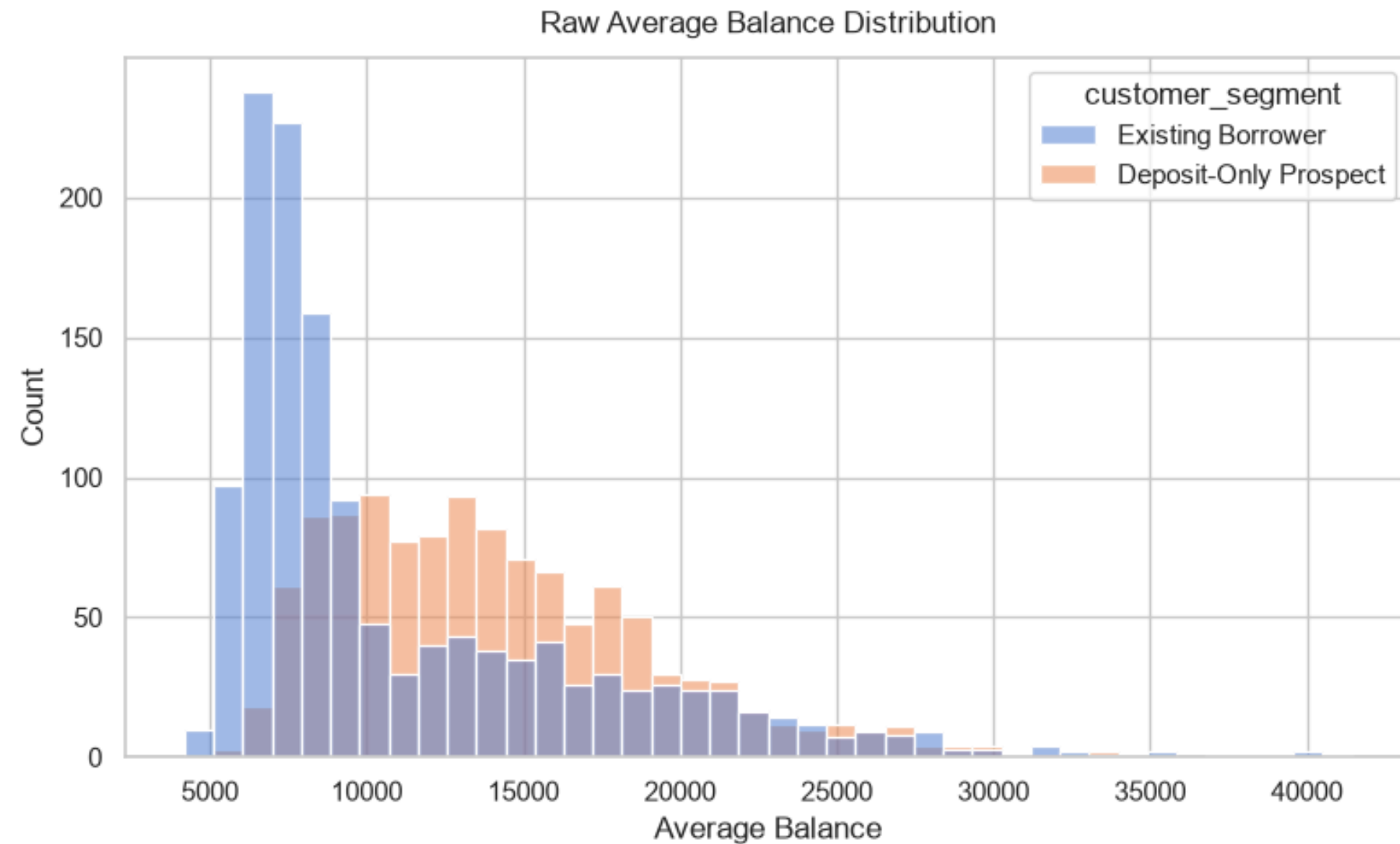
What are the Demographics?



- Our customers consists of a fair distribution of male and female who earns ~R10000 and mostly are youth as age peaks around 35 years.



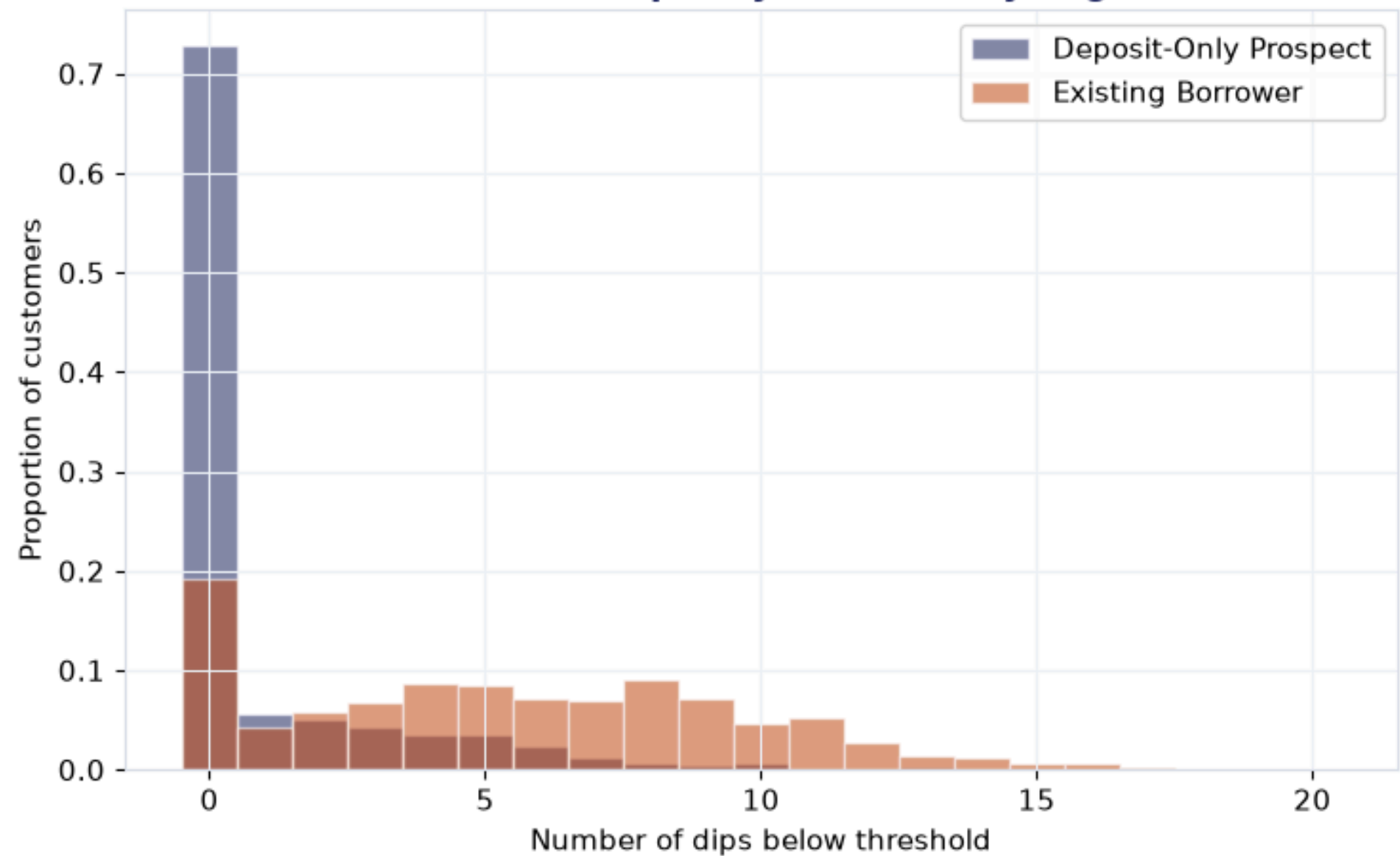
What is the disposable cash our customers have?



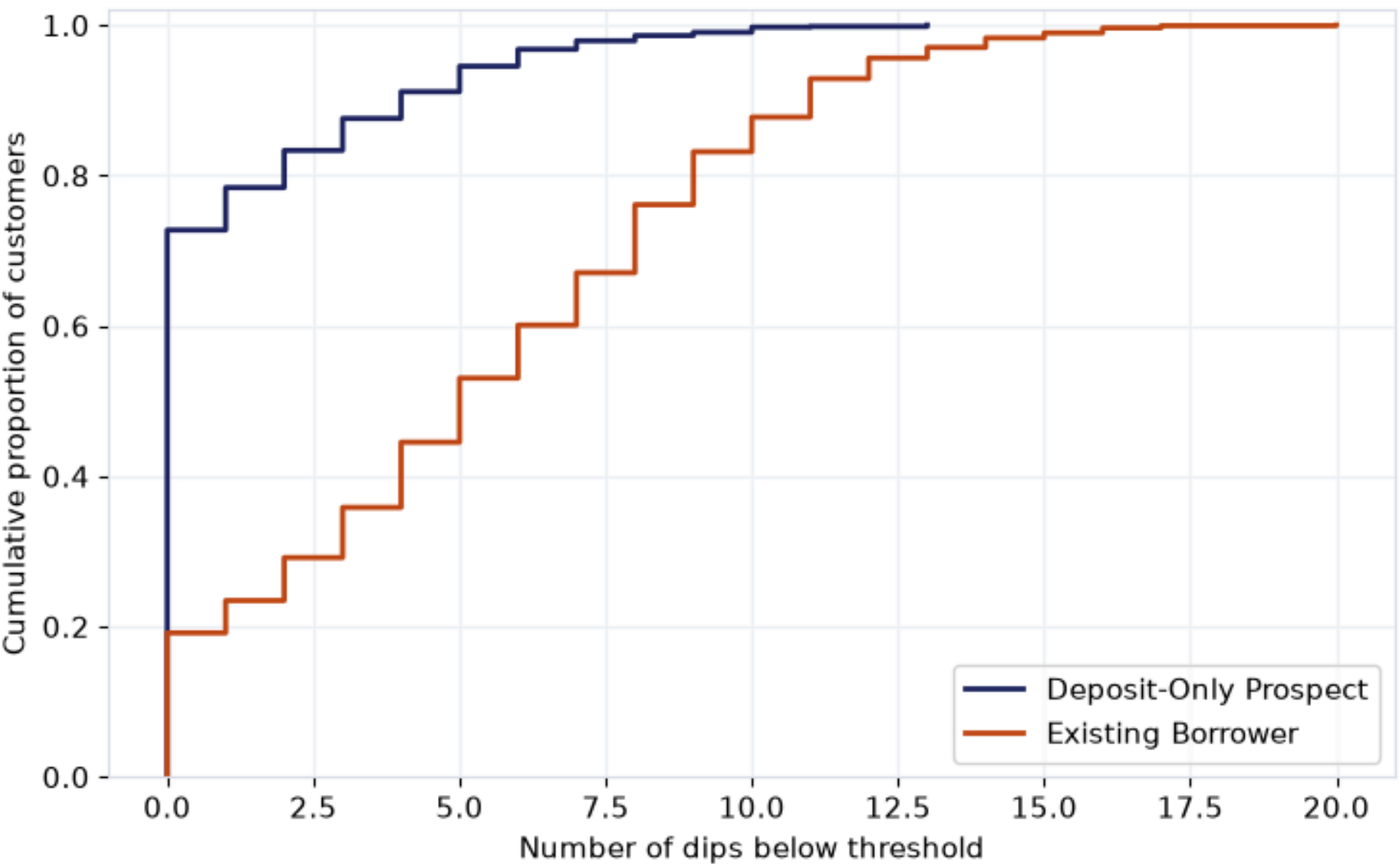
- We see that most of existing borrower customers have disposable cash of ~R8000 and only around 10% of our customers have higher disposable cash ~R25000
- When we zoom in, we further see that the deposit-only prospect customers have the same disposable cash as the existing borrowers customers at the tail.

How often does the balance go to negative status?

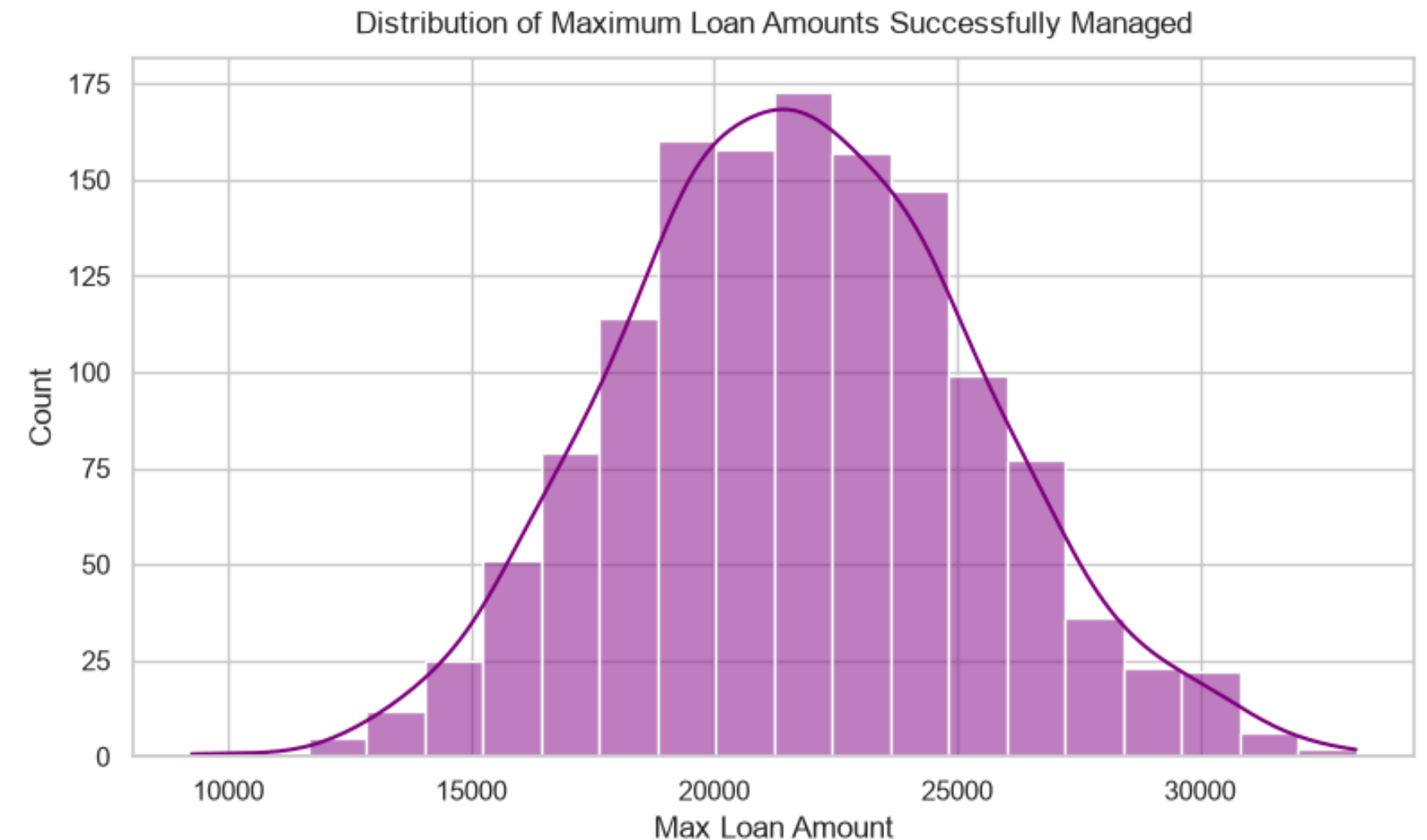
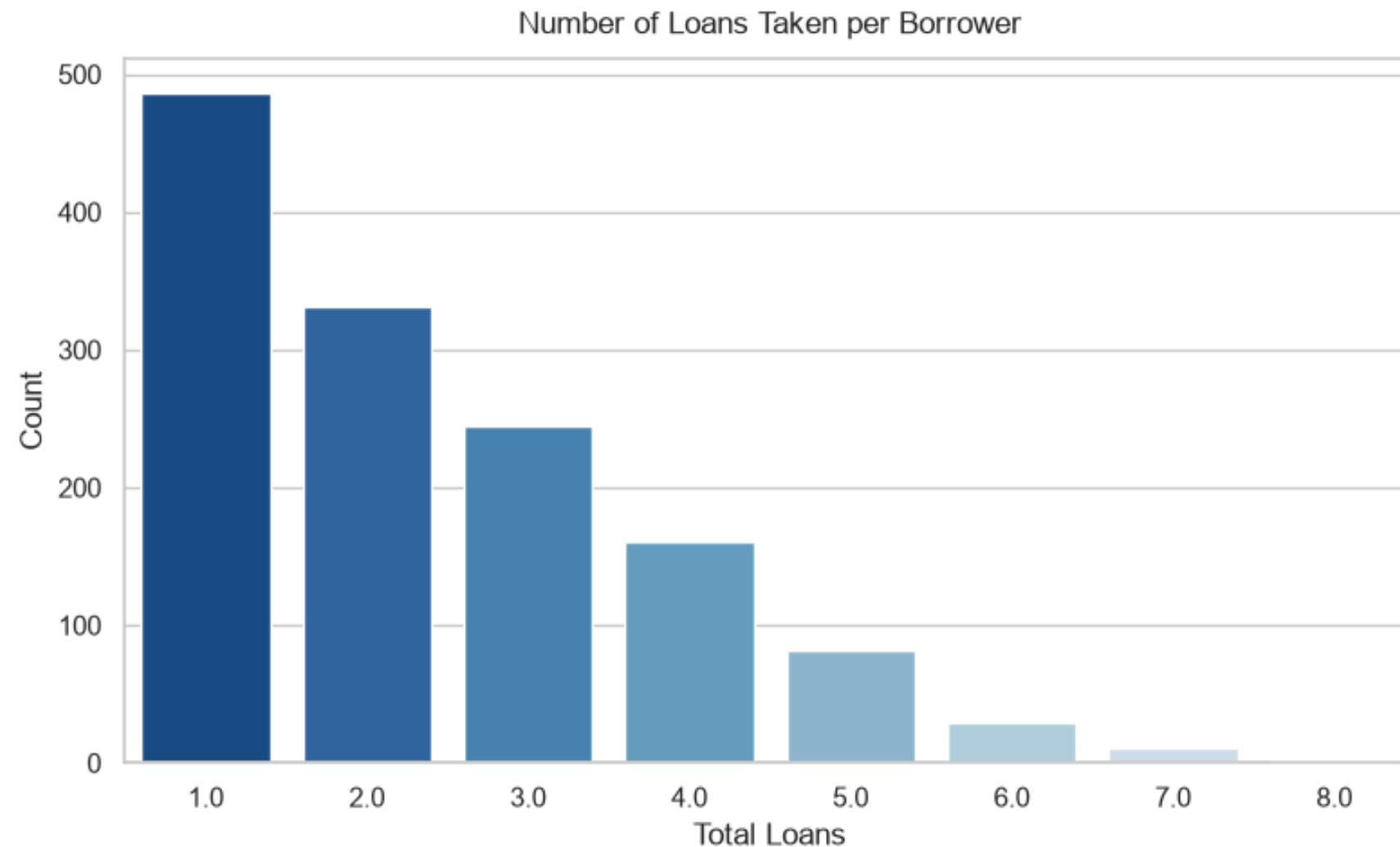
Distribution of Liquidity Crunches by Segment



Cumulative Distribution (ECDF)

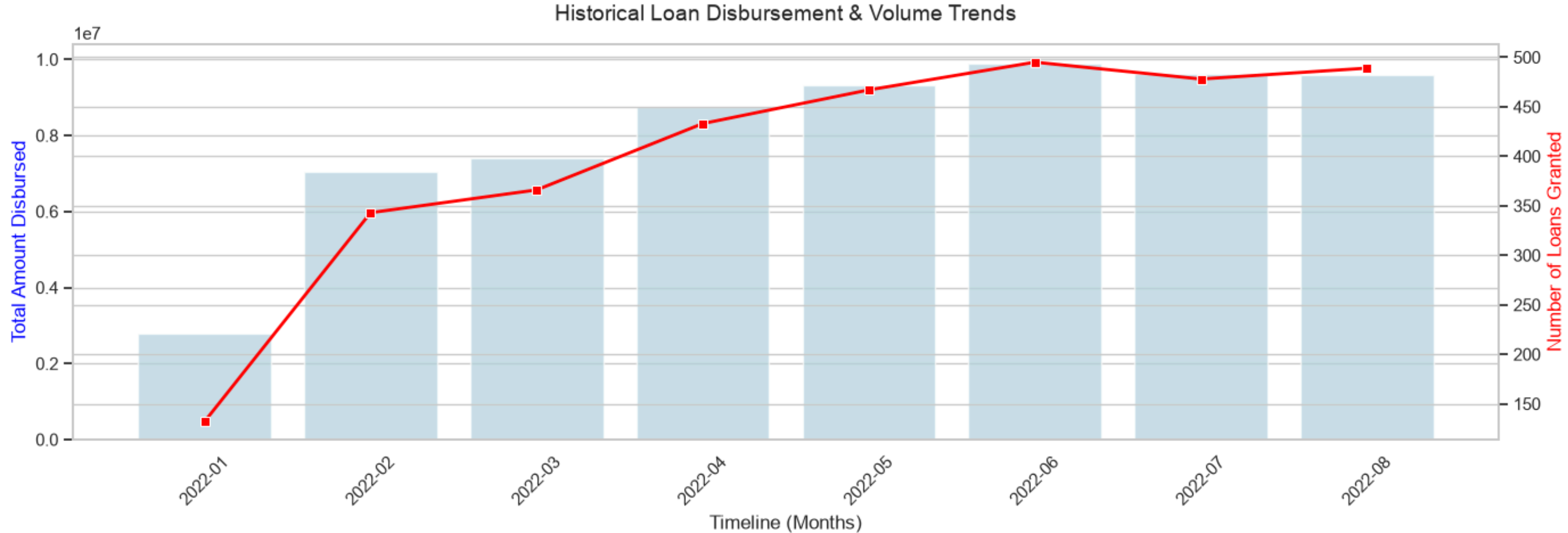
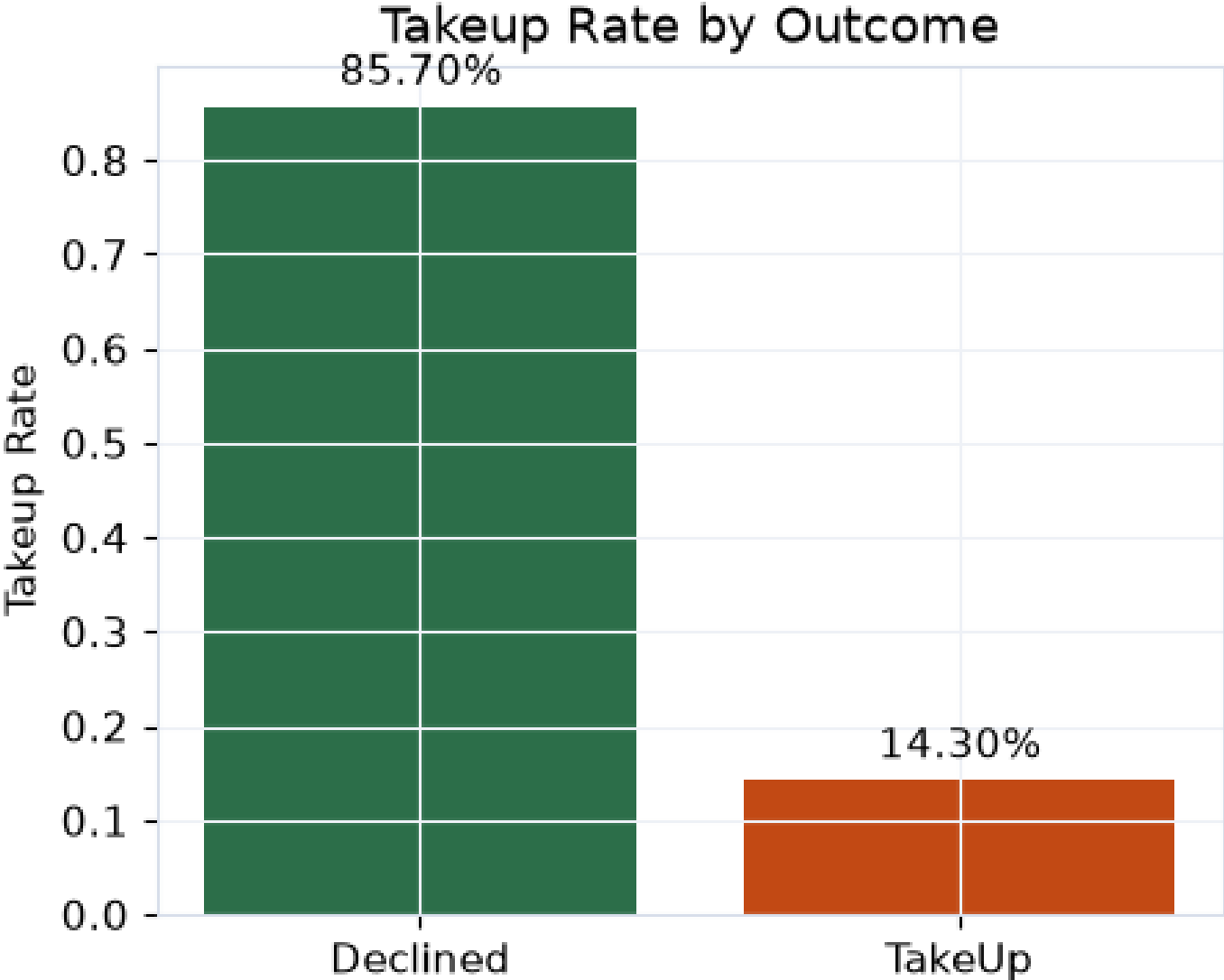


Does our customers take loans and how much?



- Most customers take up a single loan of around R21000.
- I shall use this feature because it has a normal distribution shape, to engineer KPI features

How many customers take-up a loan and what impact does it have on business?



Even though it looks like there is less number of people taking up the loan but there is a increase on the number of loans that are granted.

KPIs behind the model



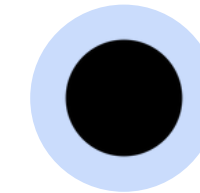
Net cash flow

Average monthly inflow minus outflow — a steady positive gap signals capacity to repay.



Balance volatility

Std. dev. of balance vs. its average — a shaky balance suggests higher near-term risk.



Burn rate

How fast balance drops after it peaks each month (a proxy for “just got paid”) — currency lost per day.



Low-balance frequency

Count of near-zero balance events — threshold set from the data's own 10th percentile, not a guessed round number.



Credit capacity

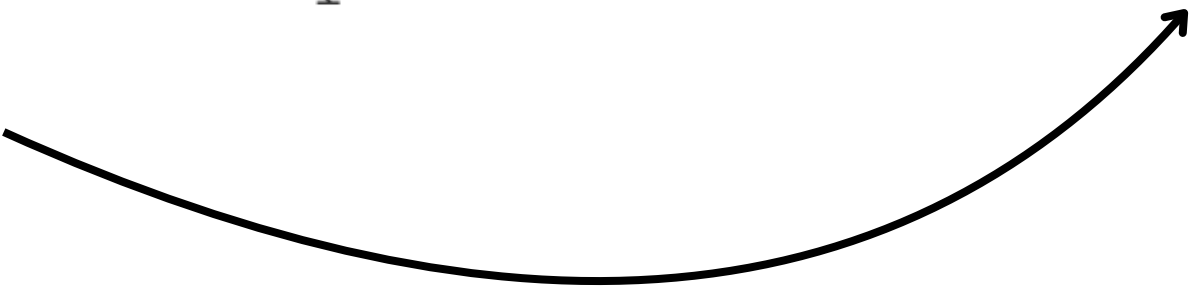
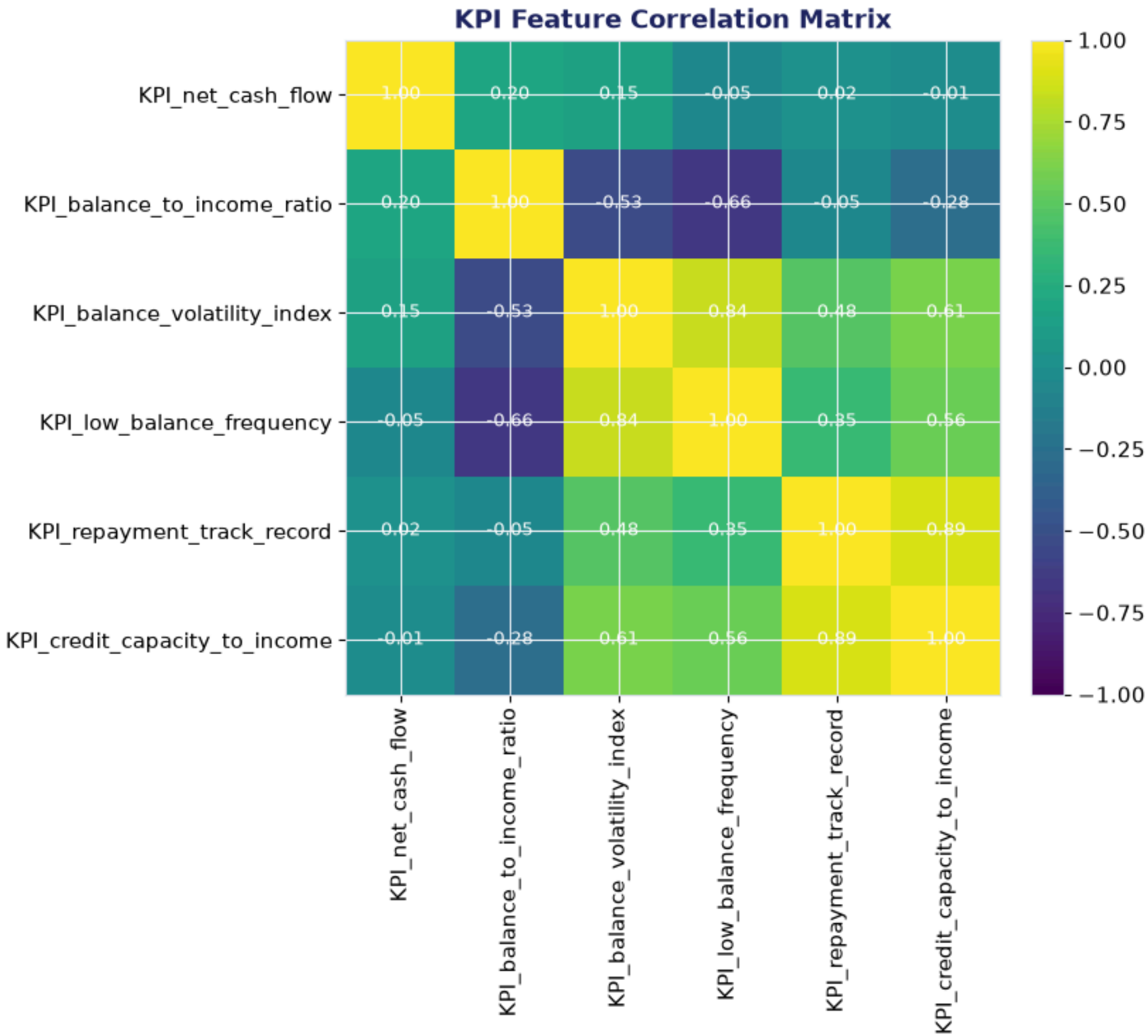
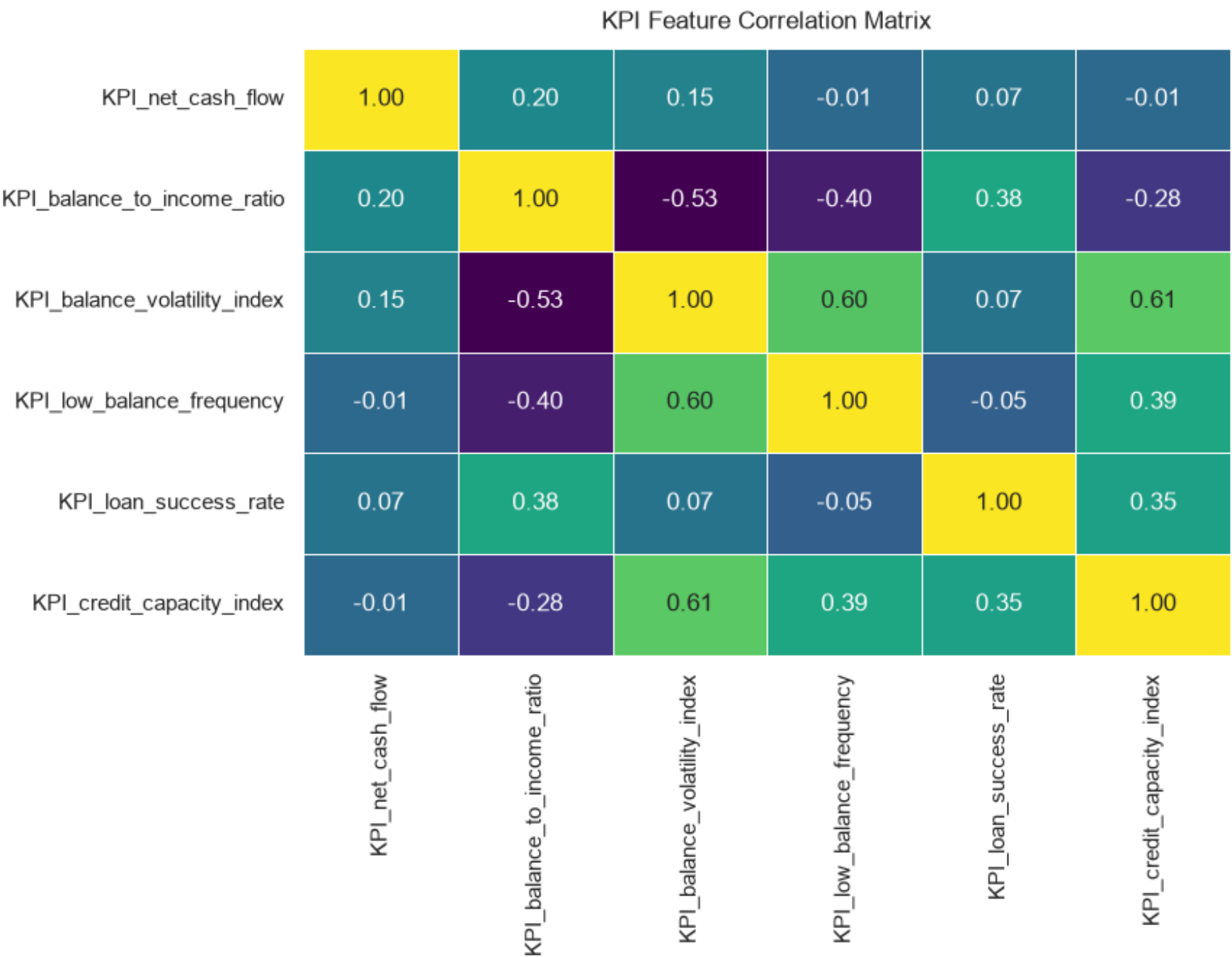
Largest amount ever successfully taken up, relative to income — a proven, not assumed, borrowing ceiling.



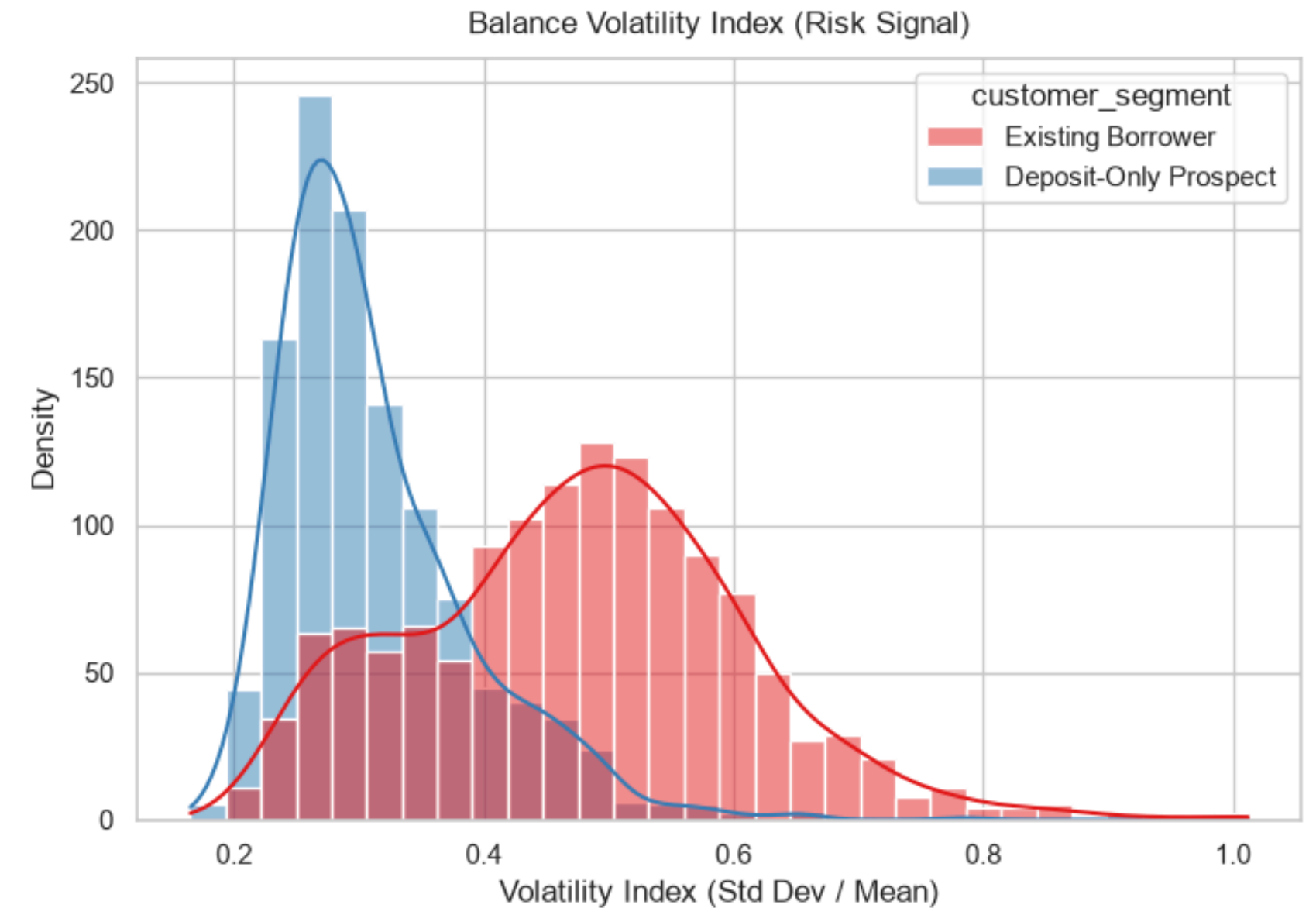
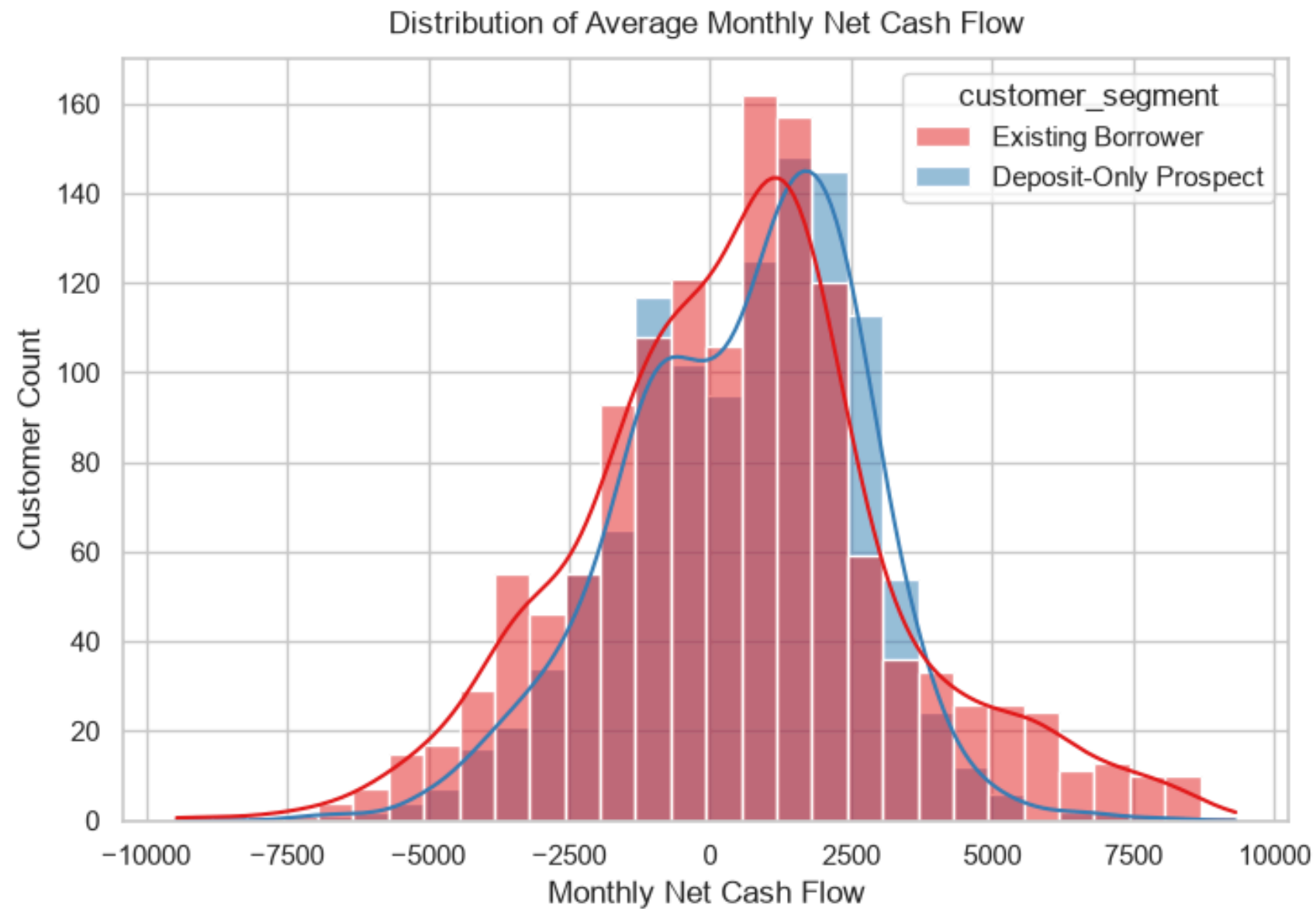
Lifecycle segment

Age × income quadrant (tertile cut-offs from the data) — young/low-income vs. mid-age/high-income vs. senior, etc.

Dealing with Collinearity !

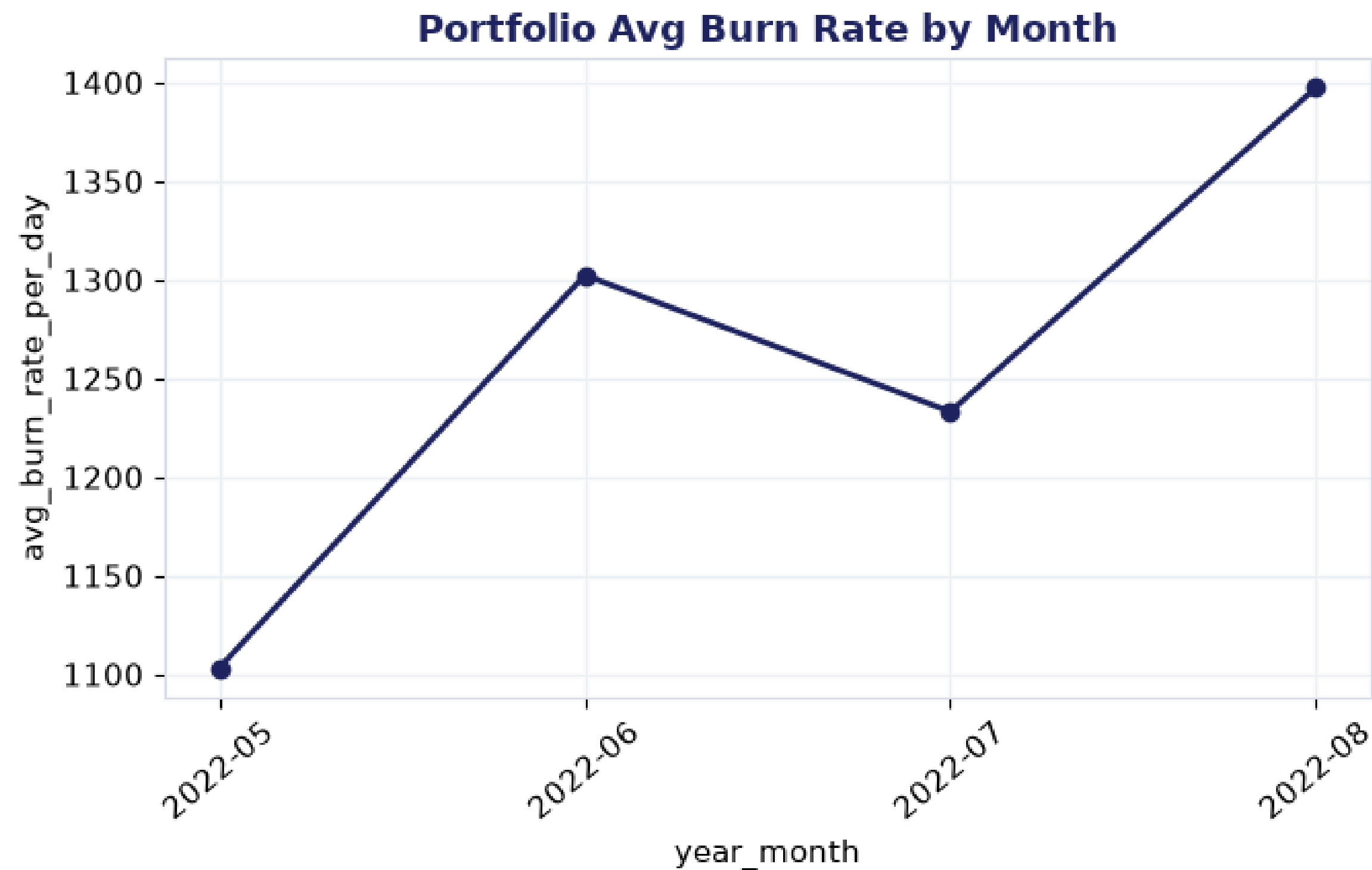


What is cash flow and balance volatility?

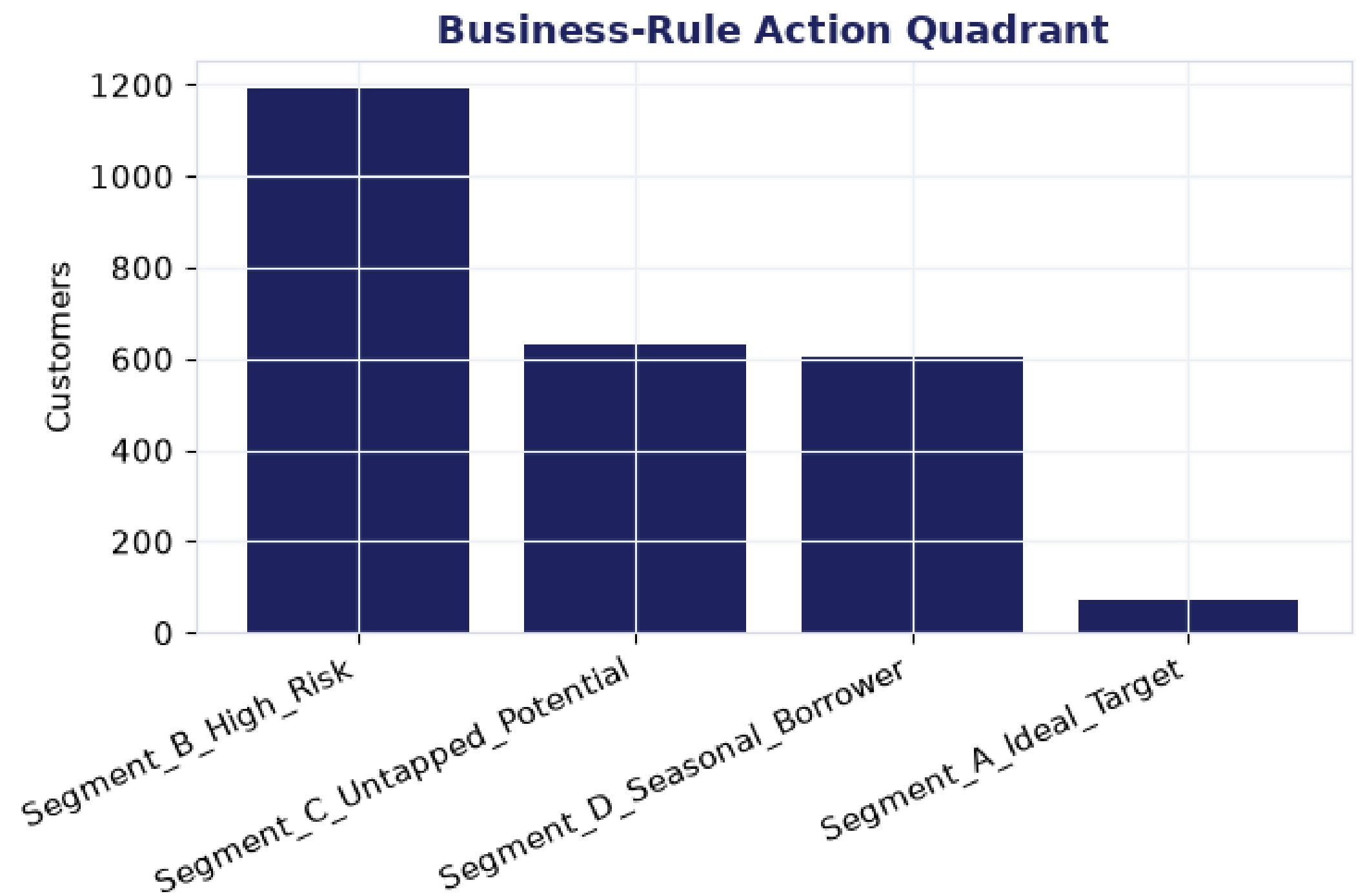


- Even though our customers have a positive cash flow, the majority of our customers - existing borrower, their balance is more volatile.

What is the burn rate of our customers

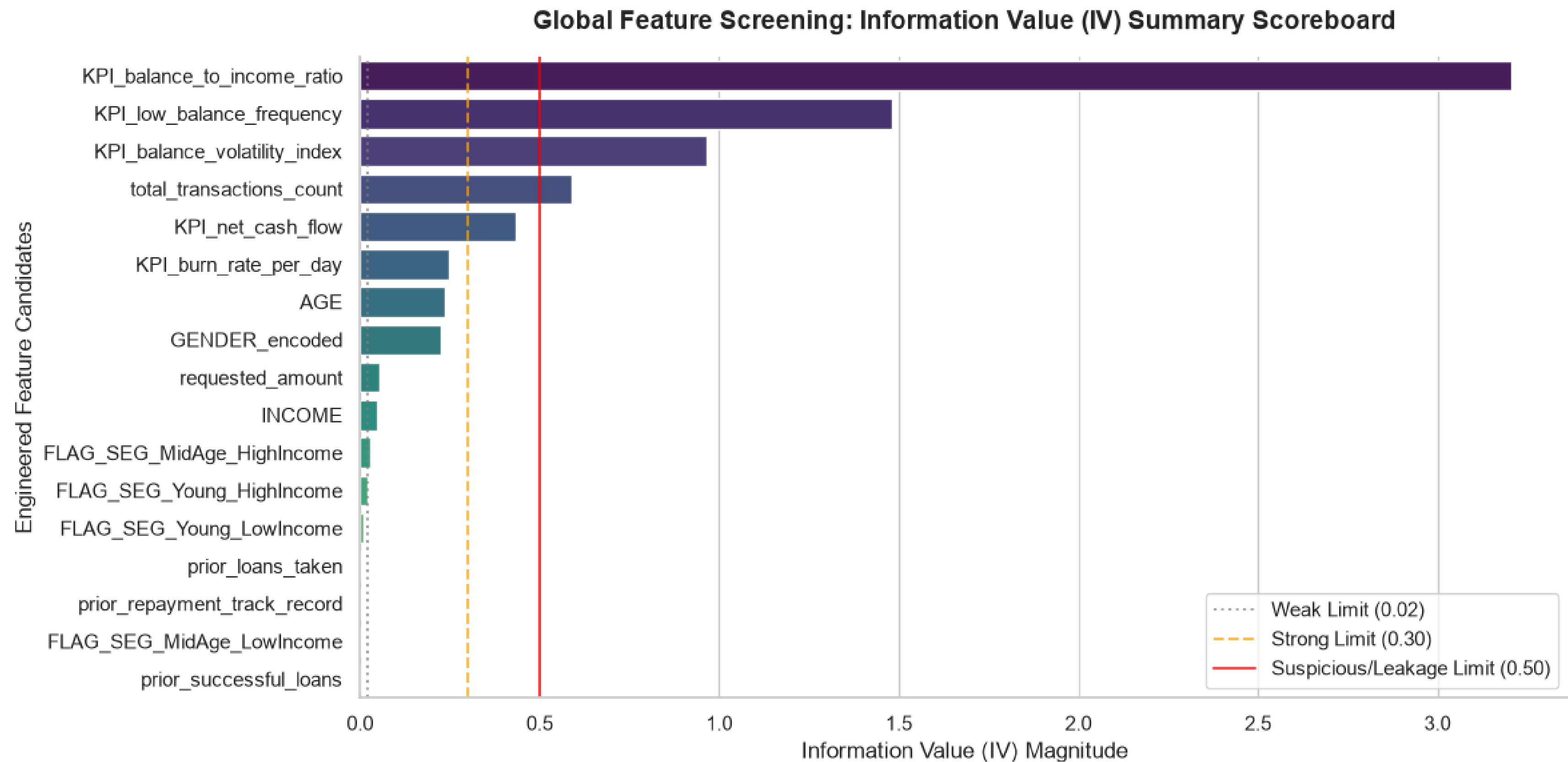


Customer Segmentation



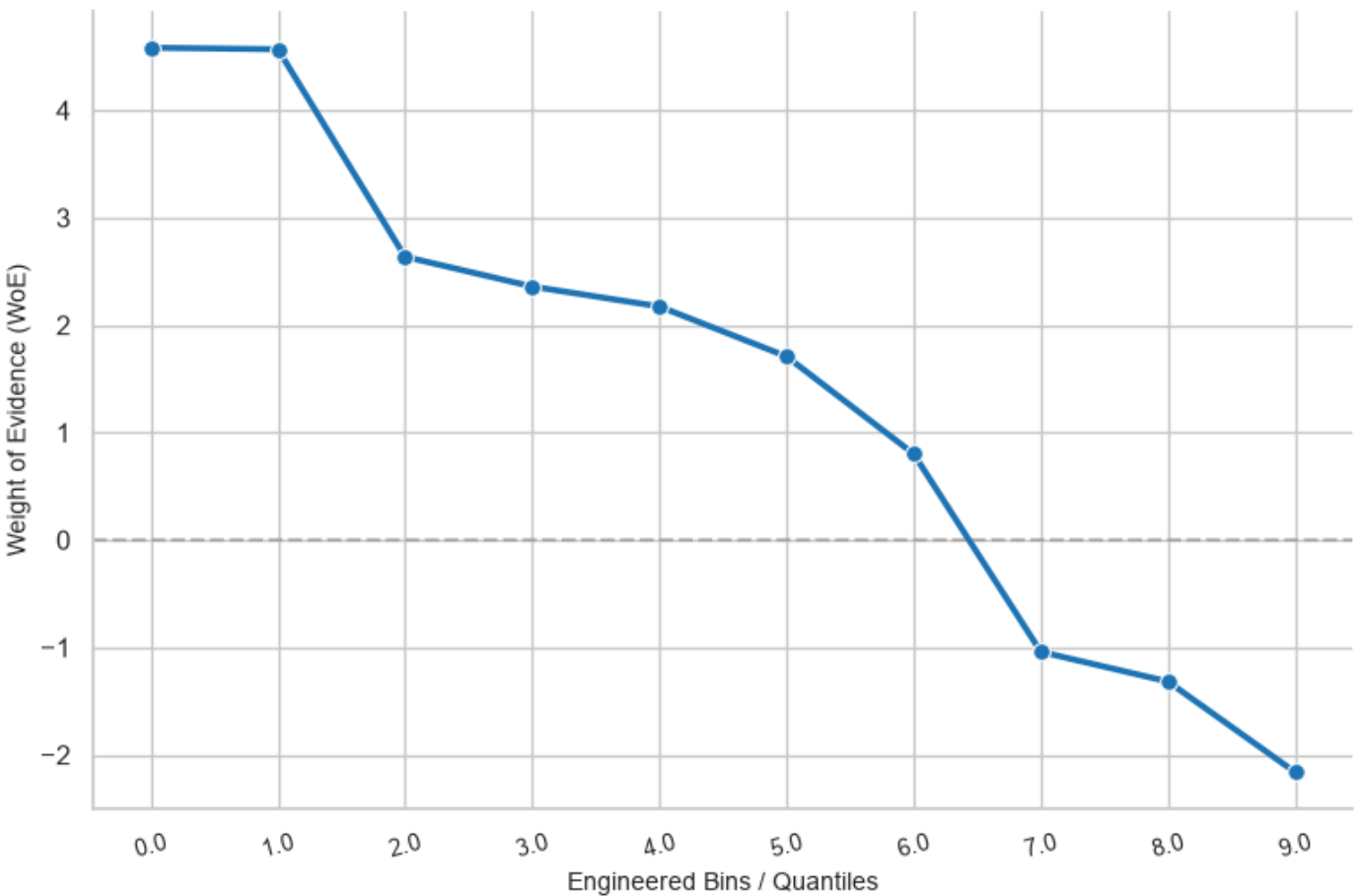
- Now we know enough about our customers and we proceed to segment them into risk using KPI's

Feature Selection

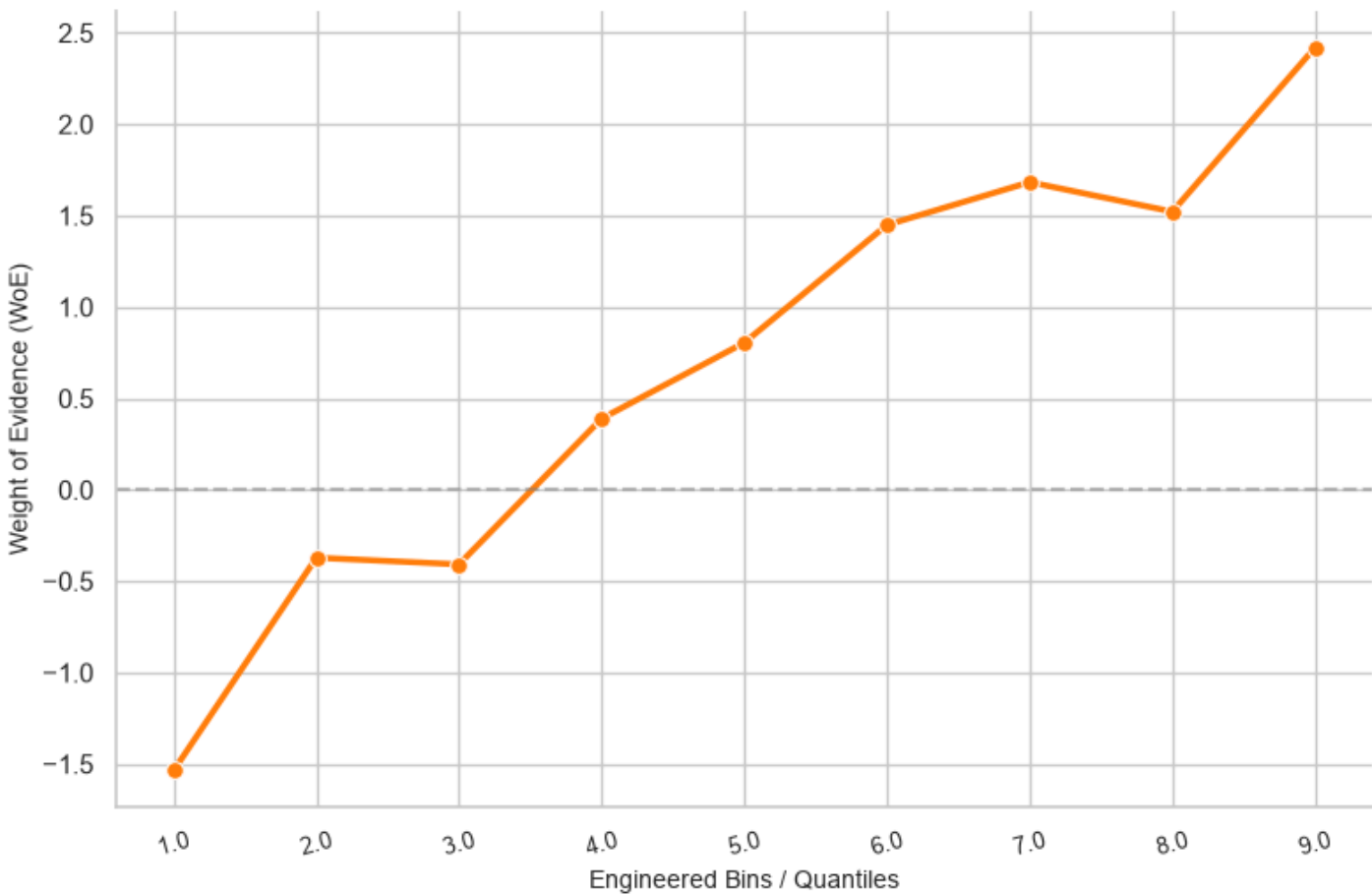


WoE: Information Value >0.50

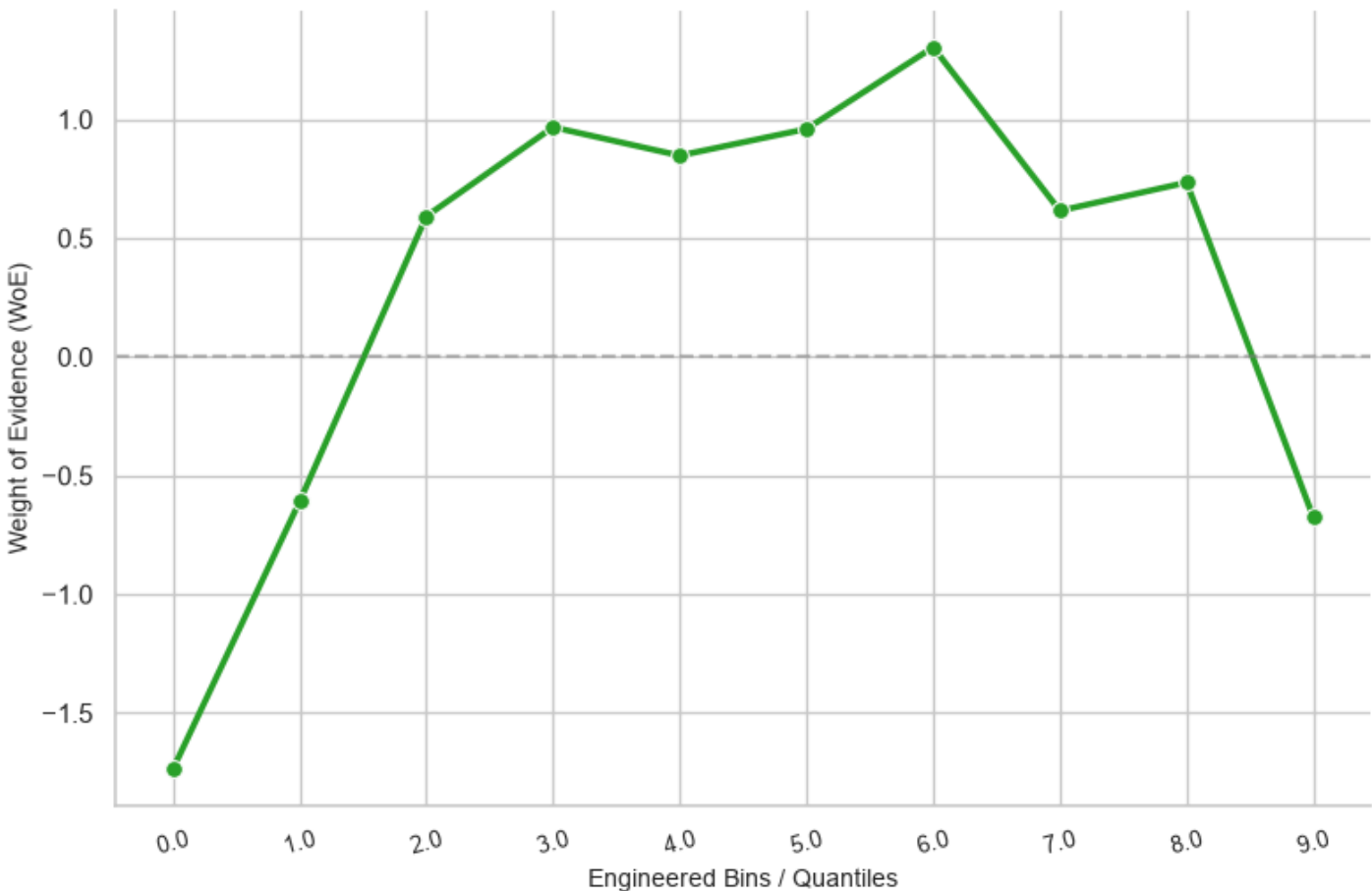
Balance-to-Income Ratio (IV: 3.205)



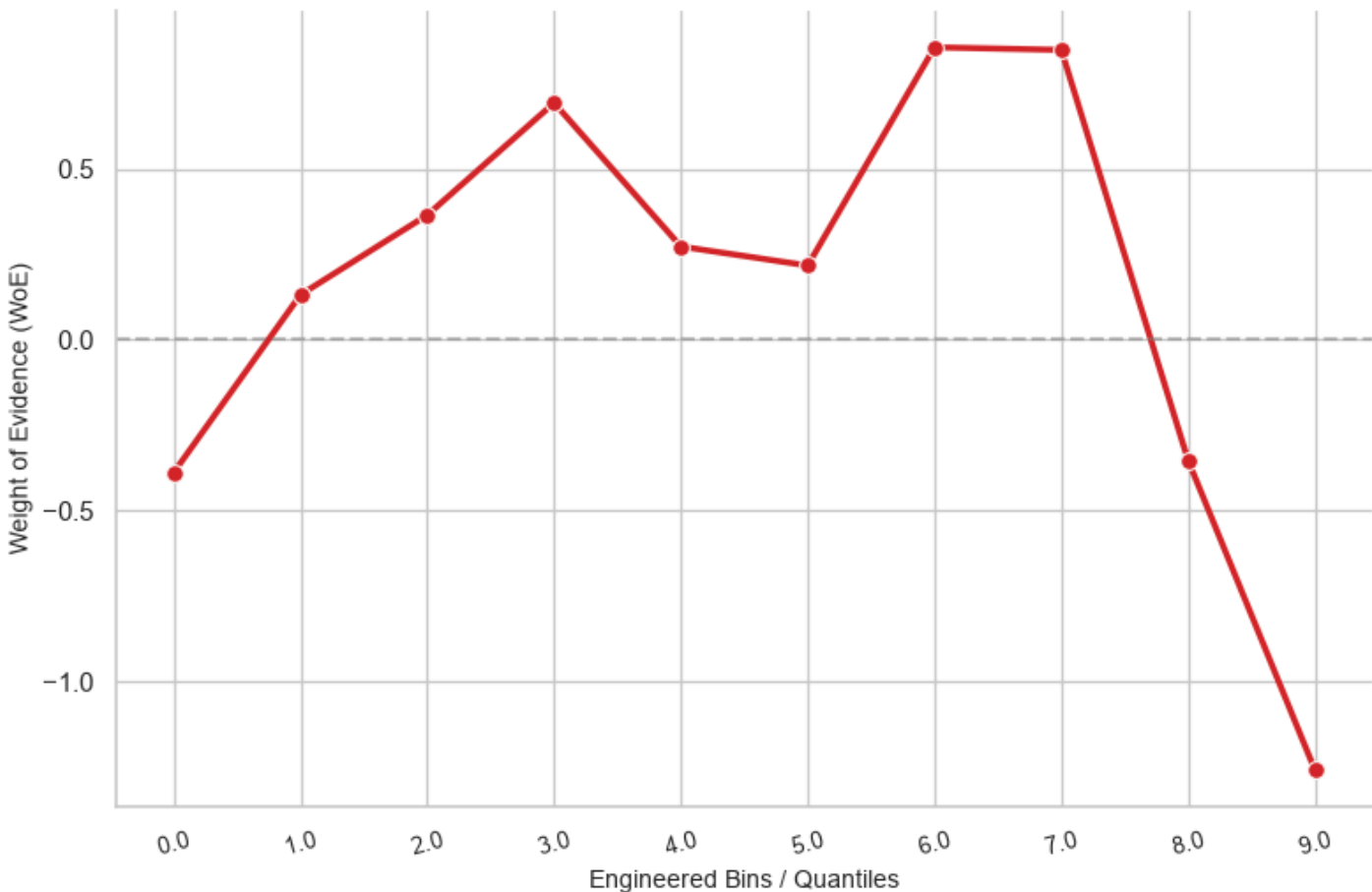
Low-Balance Frequency Crunches (IV: 1.482)



Cash Runway Volatility Index (IV: 0.966)



Monthly Net Cash Flow Margin (IV: 0.433)



Model & evaluation approach

Methodology

Algorithm:

Logistic Regression — interpretable coefficients matter more and also we used cost-sensitive learning.

Class imbalance:

Inverse-frequency row weighting (Spark ML's equivalent of scikit-learn's `class_weight='balanced'`)

Split:

Stratified 85/15 train/test, Temporal Train-Test Split to avoid feature leakage.

Grain:

One row per historical loan application — not per customer

Headline results

ROC-AUC (test set)

88%

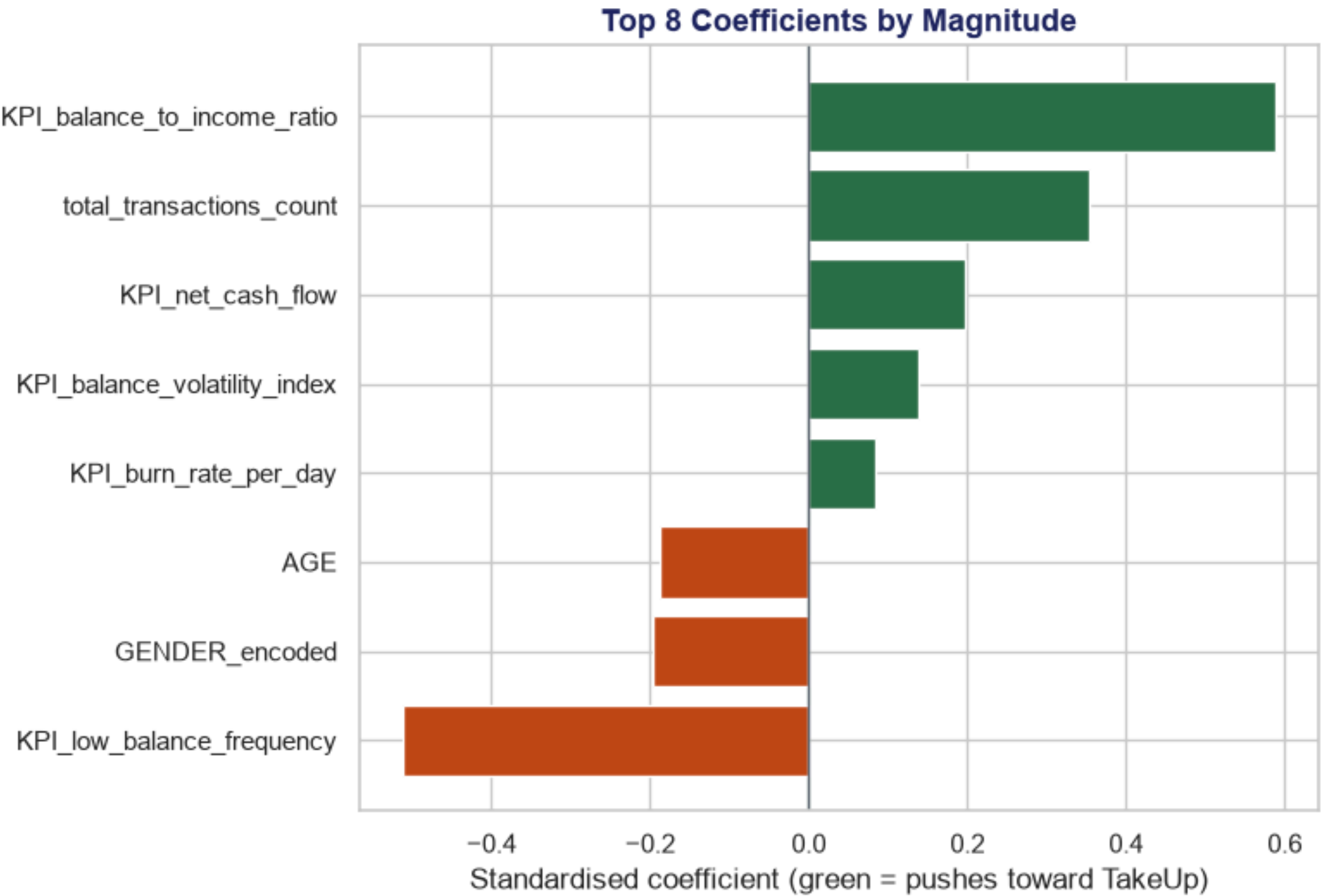
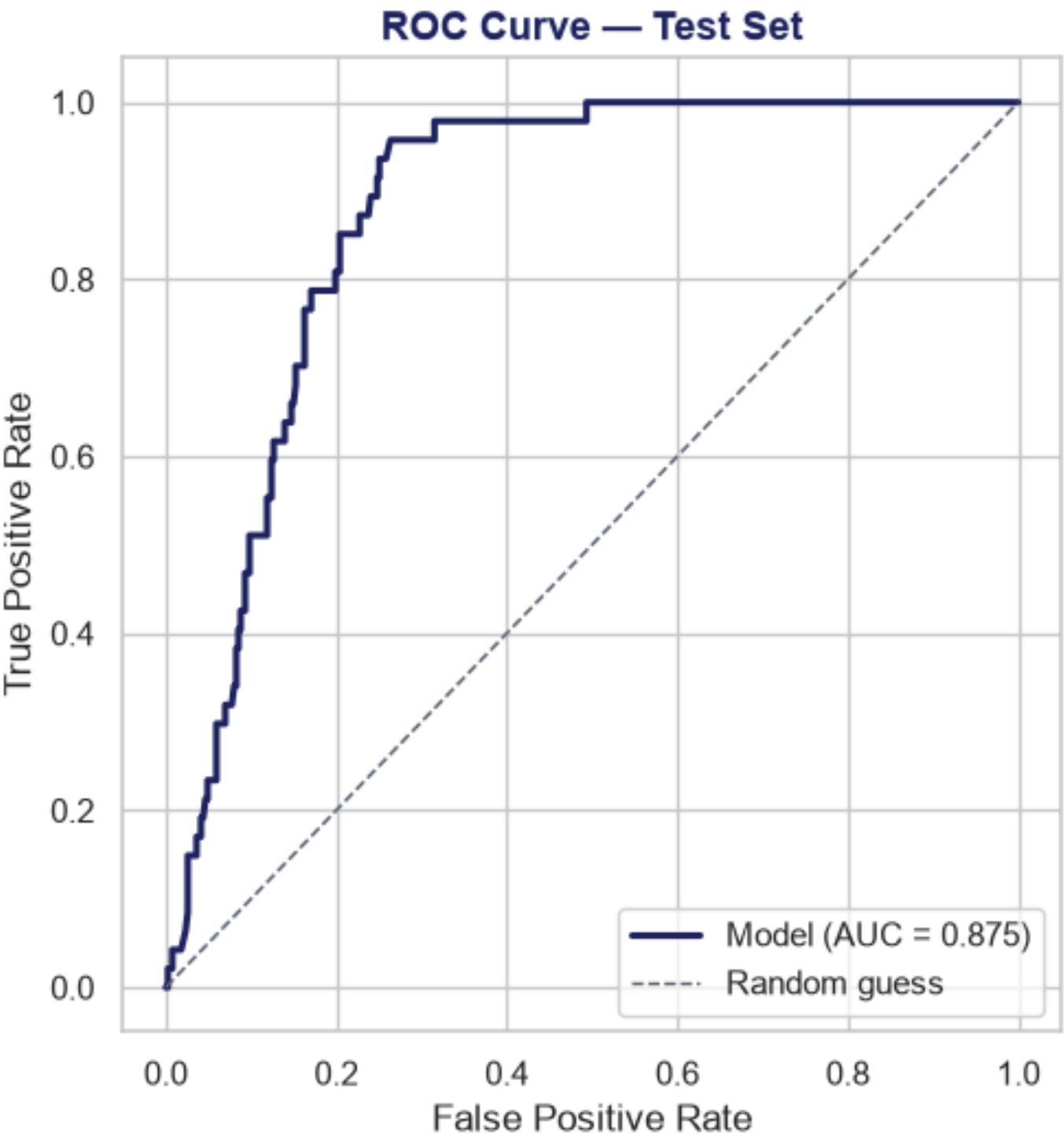
AUC without GENDER

87%

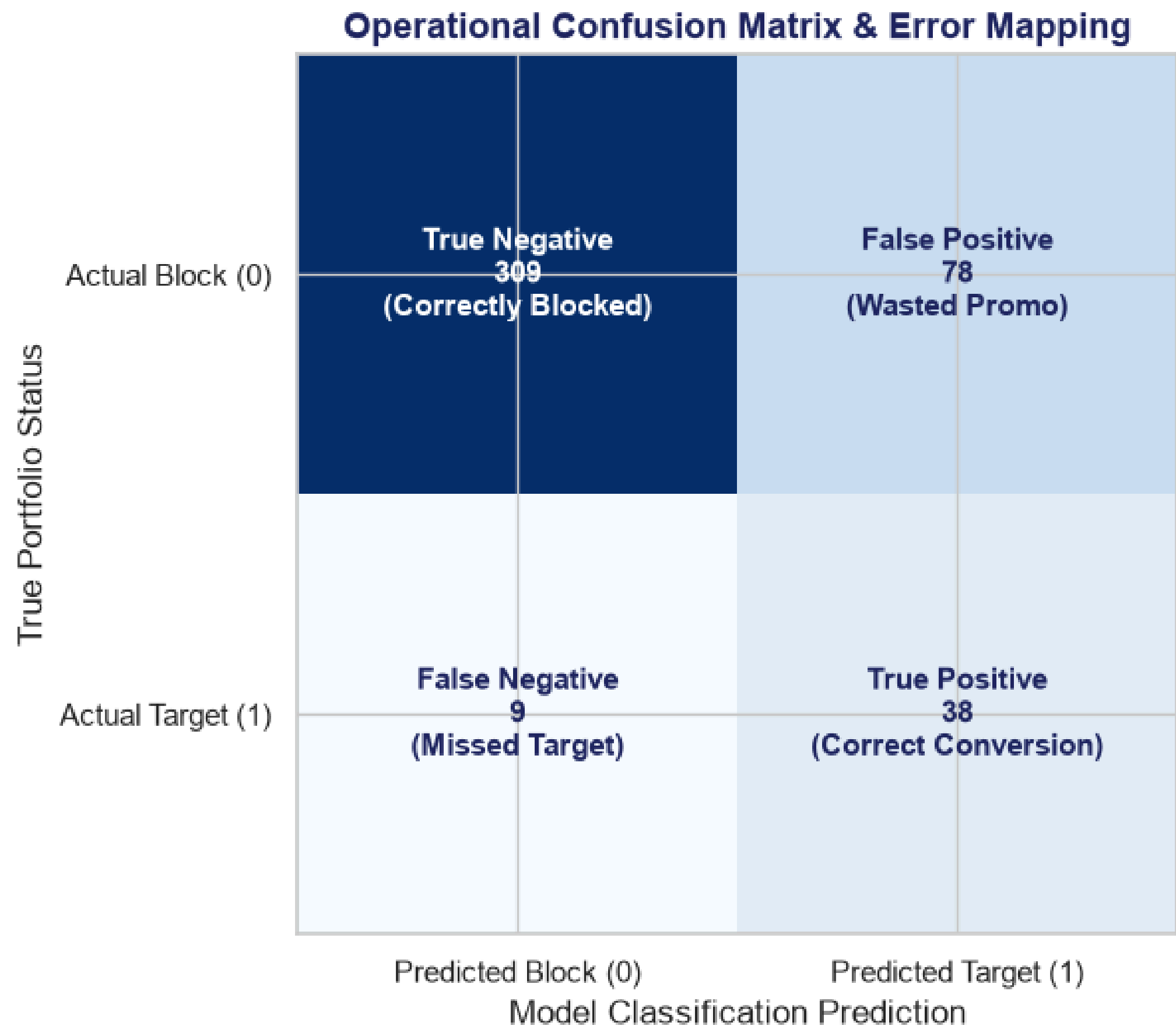
Train / test size

85/15

Model Results

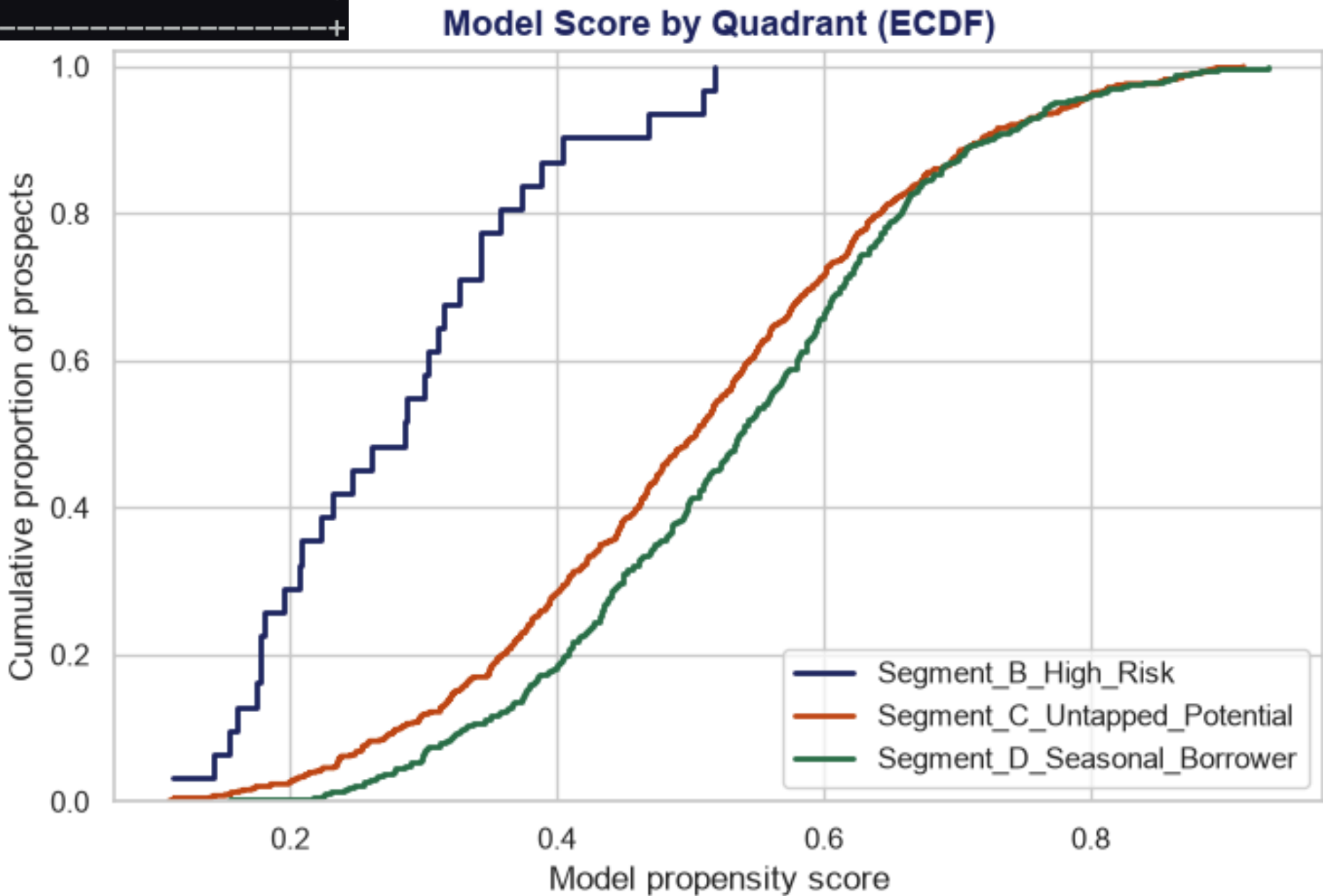


How often are we correct?

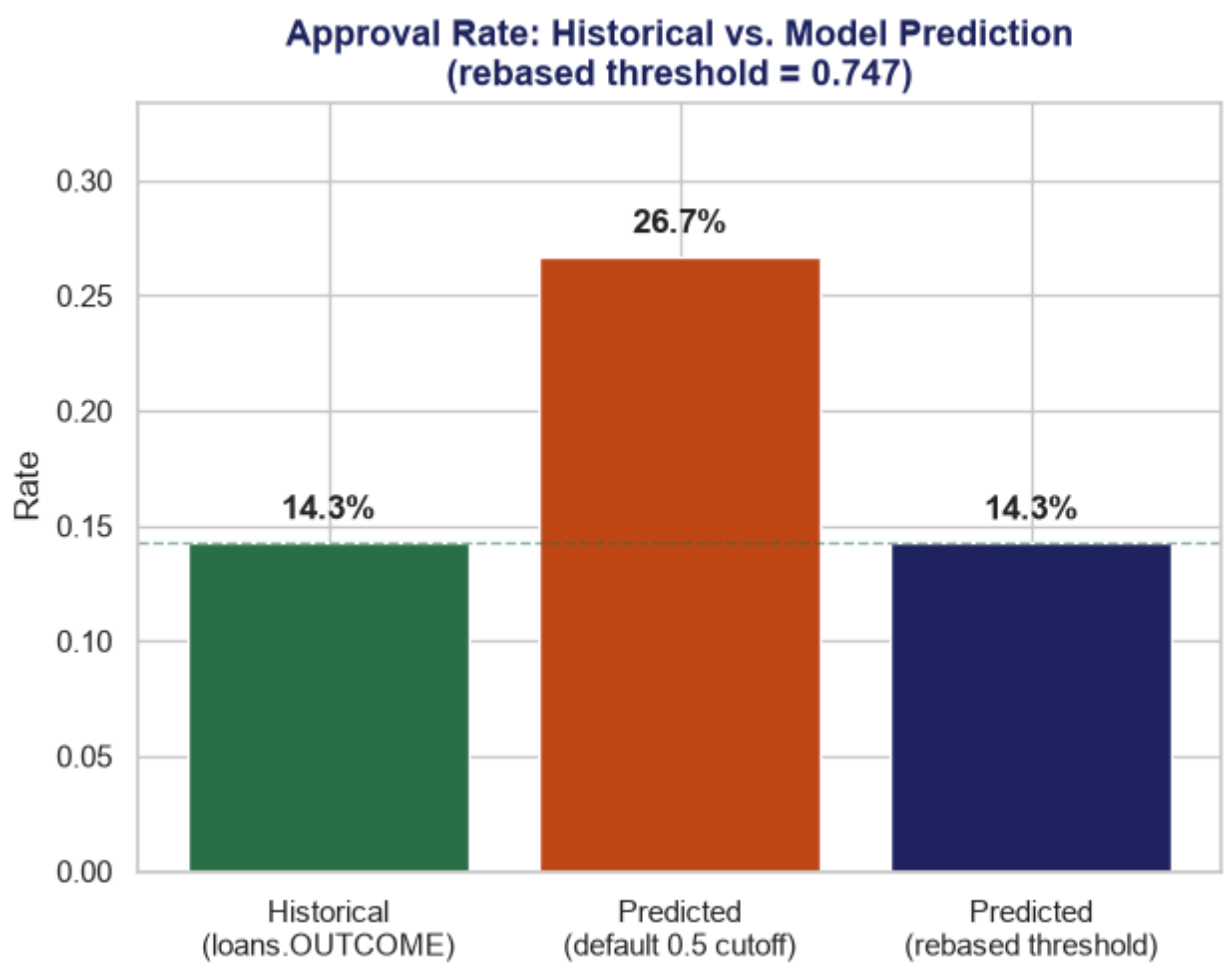
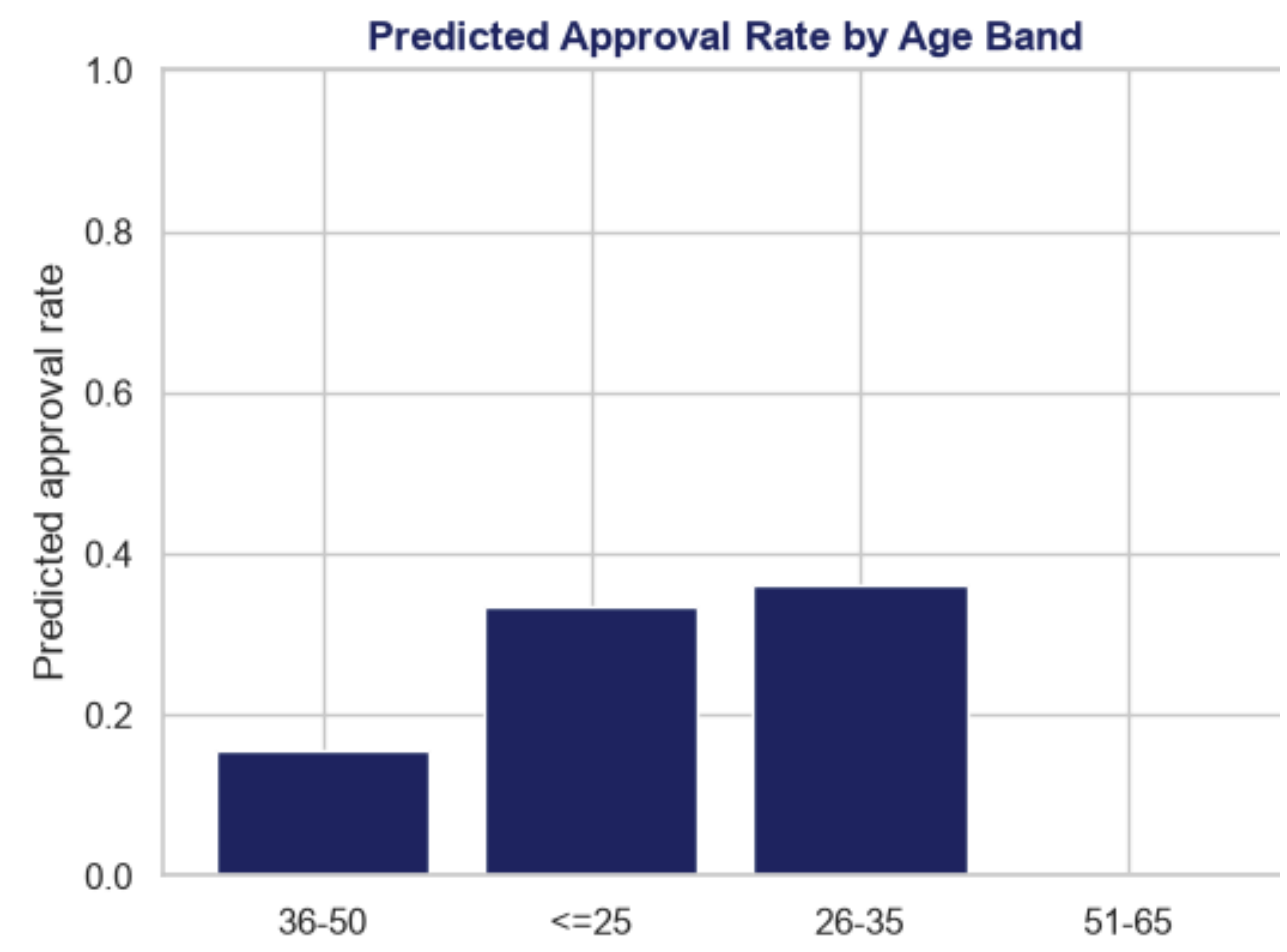
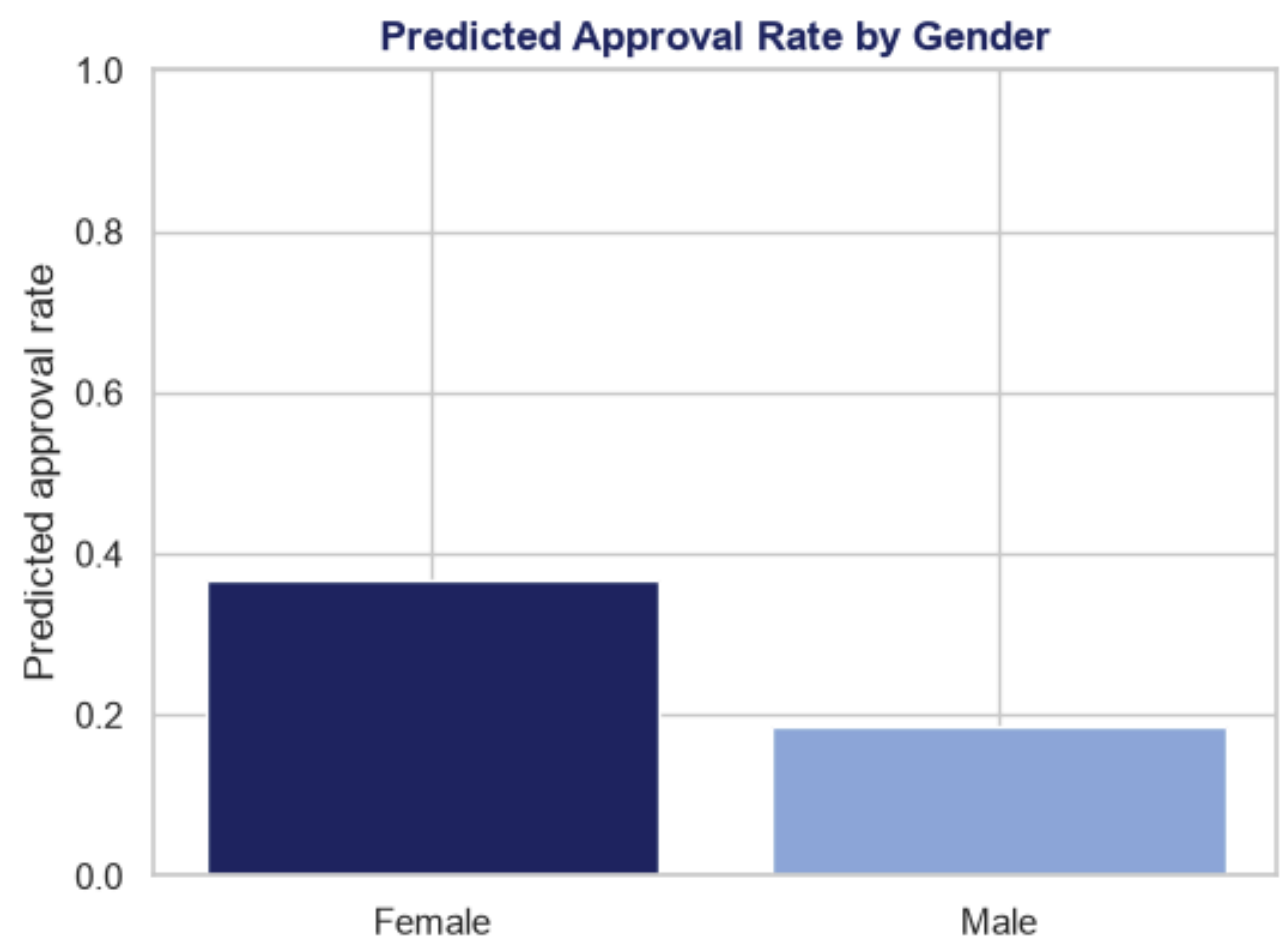


ID	customer_segment	KPI_action_quadrant	model_propensity_score
FI58QUTS	Deposit-Only Prospect	Segment_D_Seasonal_Borrower	0.9333490136914939
SHLWYLW0	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.9145840563885834
RJR0L9NE	Deposit-Only Prospect	Segment_D_Seasonal_Borrower	0.9017157732589922
GIDHRFQ2	Deposit-Only Prospect	Segment_D_Seasonal_Borrower	0.8950421111160881
TAZDLD5N	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.8947567424138905
WKYSXYK6	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.8921102217256023
/XSNIWQ8	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.8885480358239097
LAZ2IB6N	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.8859059514545173
QXJDJKT	Deposit-Only Prospect	Segment_C_Untapped_Potential	0.8826080268807563
N1ZIXCUS	Deposit-Only Prospect	Segment_D_Seasonal_Borrower	0.8813033467182415

Identified List



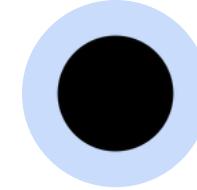
Is there Bias in the Data?



Fairness Report



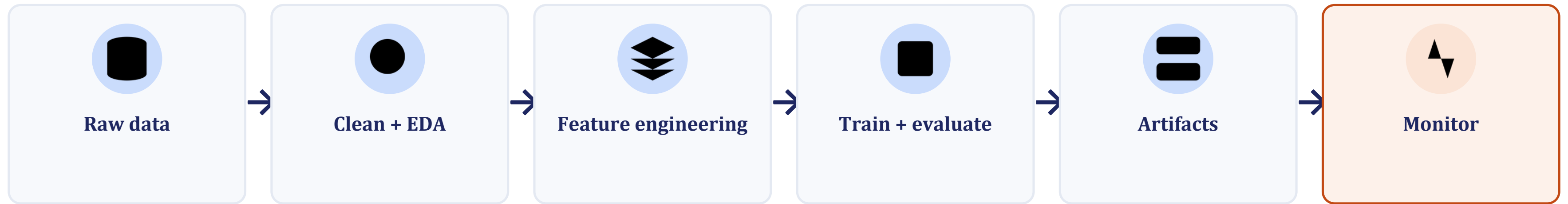
1. Disparate impact



2. Does gender earn its place?

- disparate_impact_ratio_gender: 0.50
- 'Gender' adds little to no measurable predictive power here;
 - excluding it from the live model costs little and removes a protected characteristic.

How this runs end to end



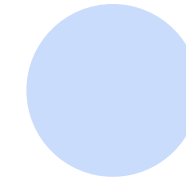
GitHub : <https://github.com/Chuene100/Loan-Propensity-Model>

What we watch once this is live



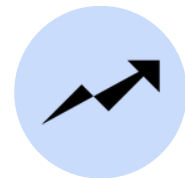
Prediction volume & latency

Every score call is counted and timed — catches a serving outage or a slow-down before a downstream team notices.



Score distribution

A histogram of propensity scores over time — a sudden shift (everyone scoring high, or low) is often the first sign something upstream changed.



Feature drift (PSI)

Population Stability Index compares today's feature distributions to training — <0.10 stable, $0.10\text{--}0.25$ monitor, >0.25 consider retraining.



Fairness over time

The disparate-impact check from slide 22 isn't a one-off — re-run on a schedule so a shift in approval-rate gaps gets caught, not just checked once at launch.

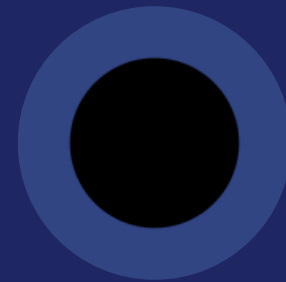


Prometheus + Grafana

The serving app exposes a `/metrics` endpoint. Grafana connects via a provisioned Prometheus data source, with a starter dashboard covering request volume, latency, error rate, and score distribution.

Recommendations & next steps

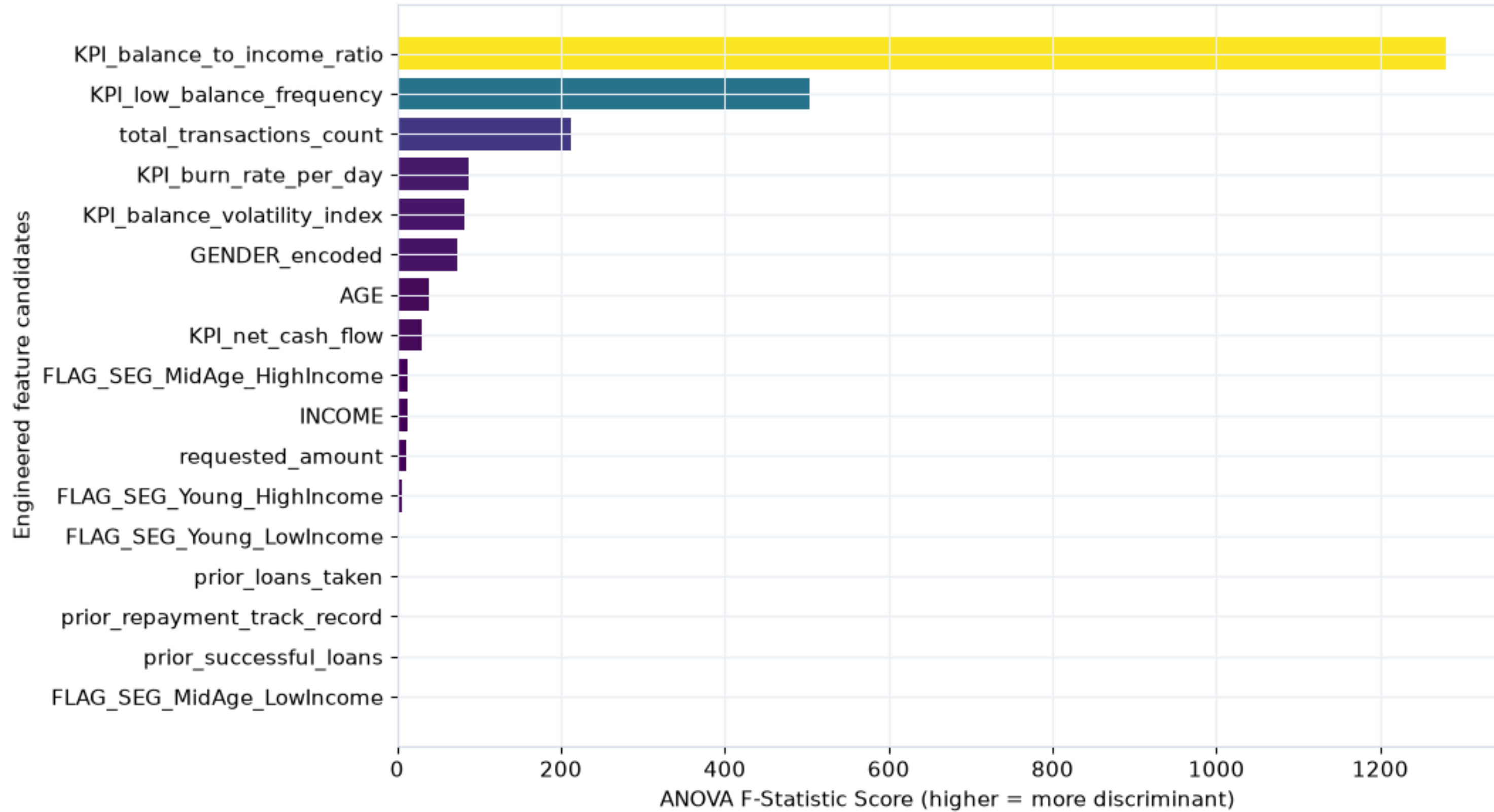
- 1** Run on the full historical dataset
- 2** Pilot on a small, monitored segment
- 3** Decide the serving path: batch scoring vs real-time endpoint
- 4** Schedule fairness & drift re-checks



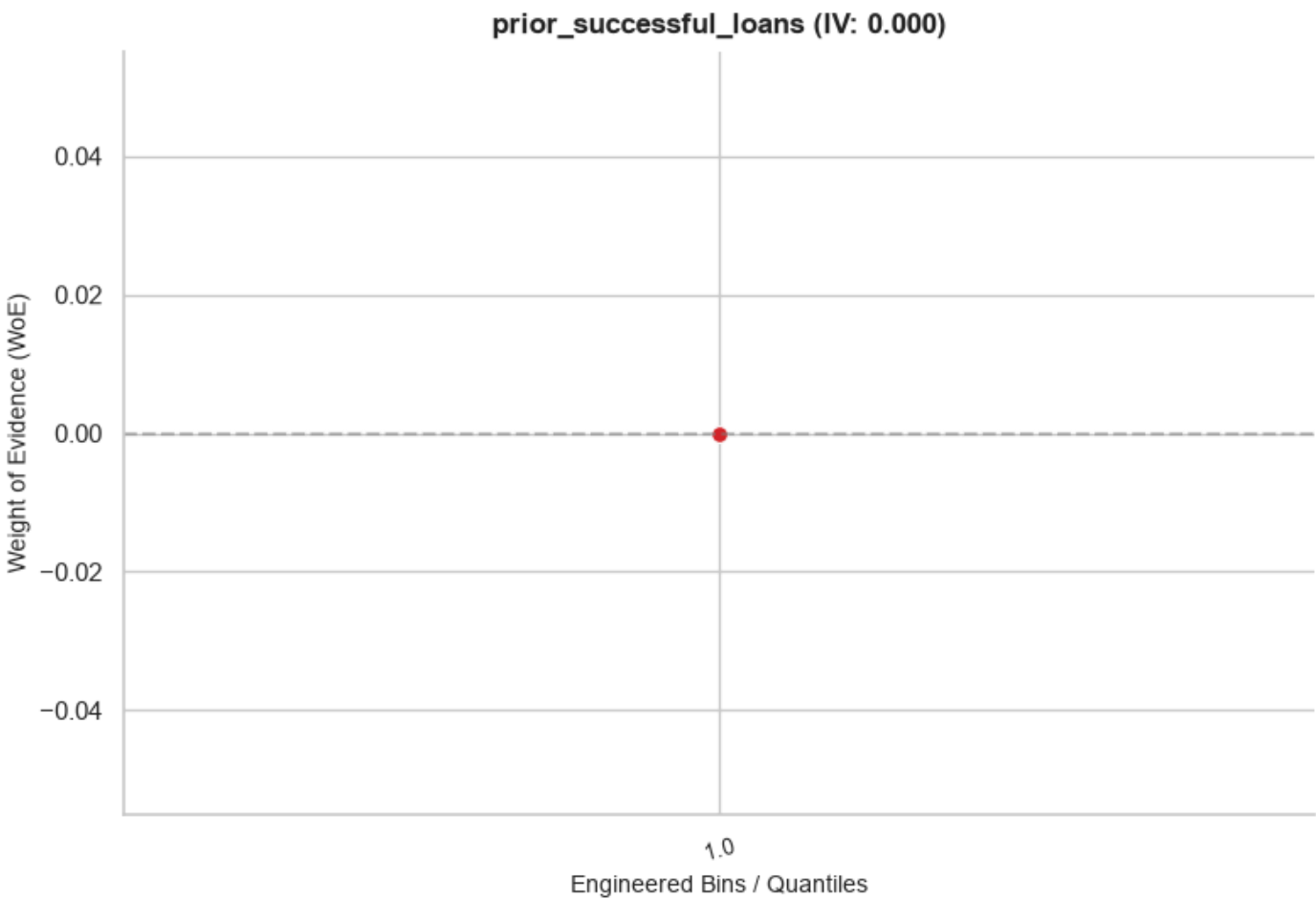
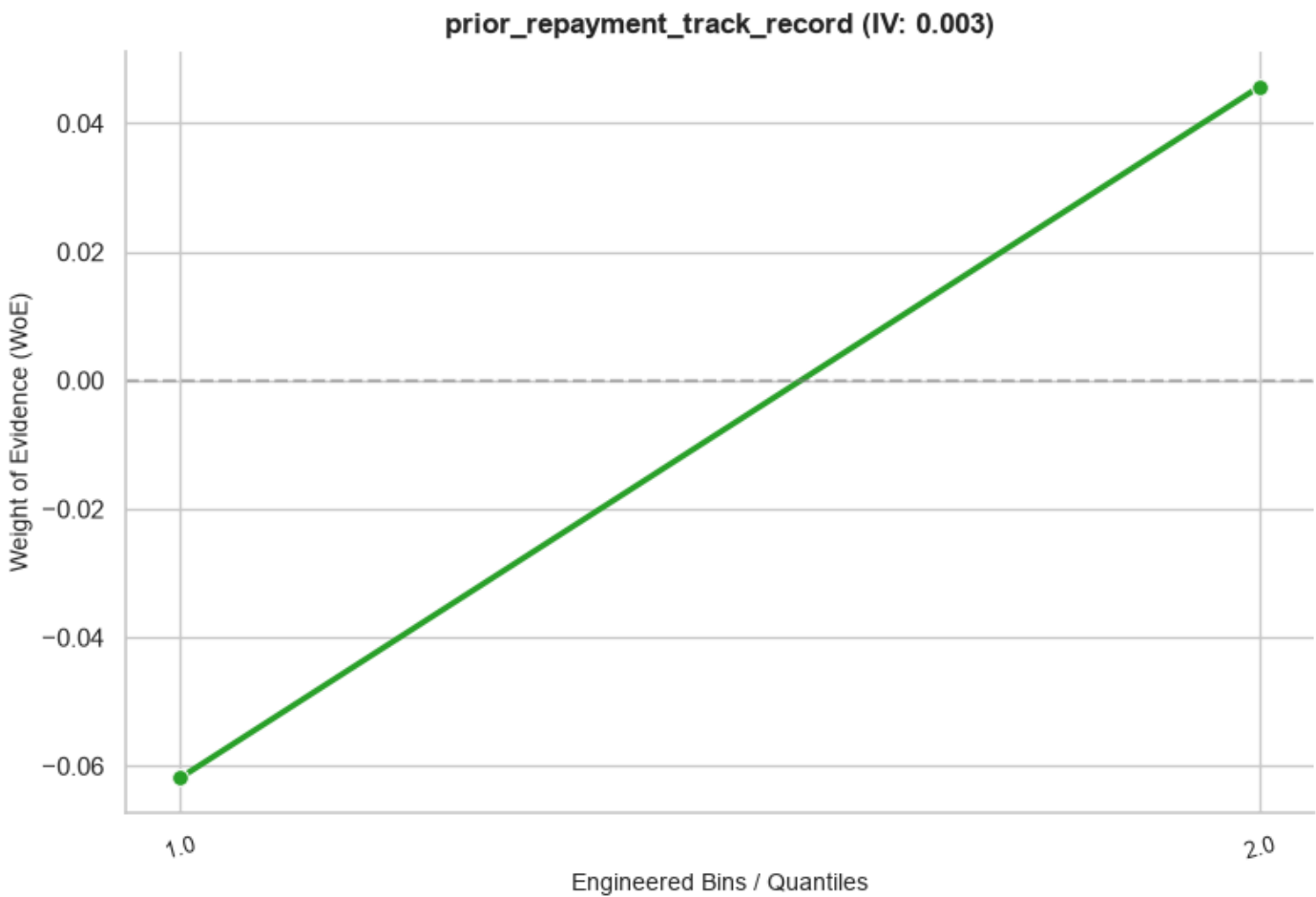
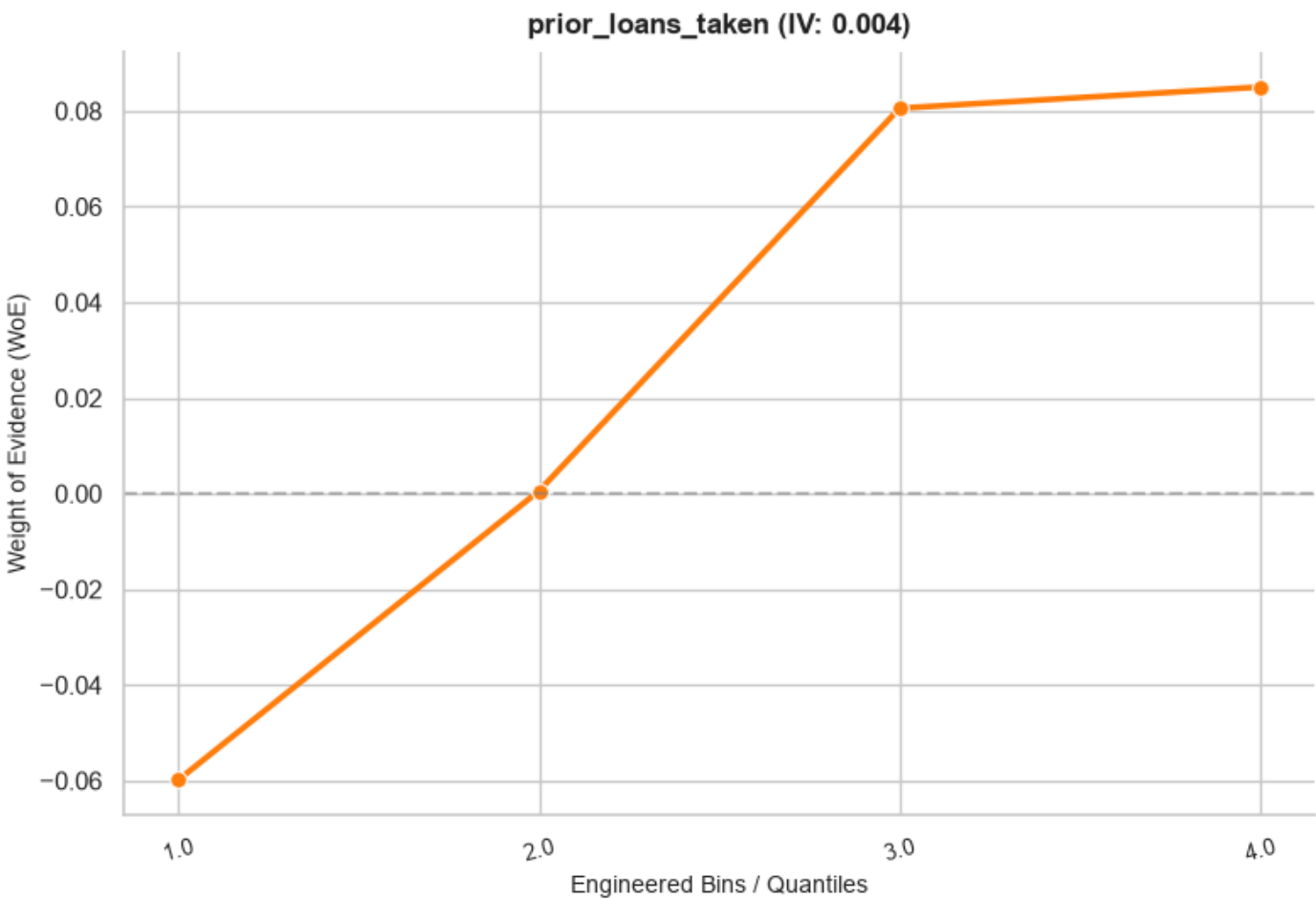
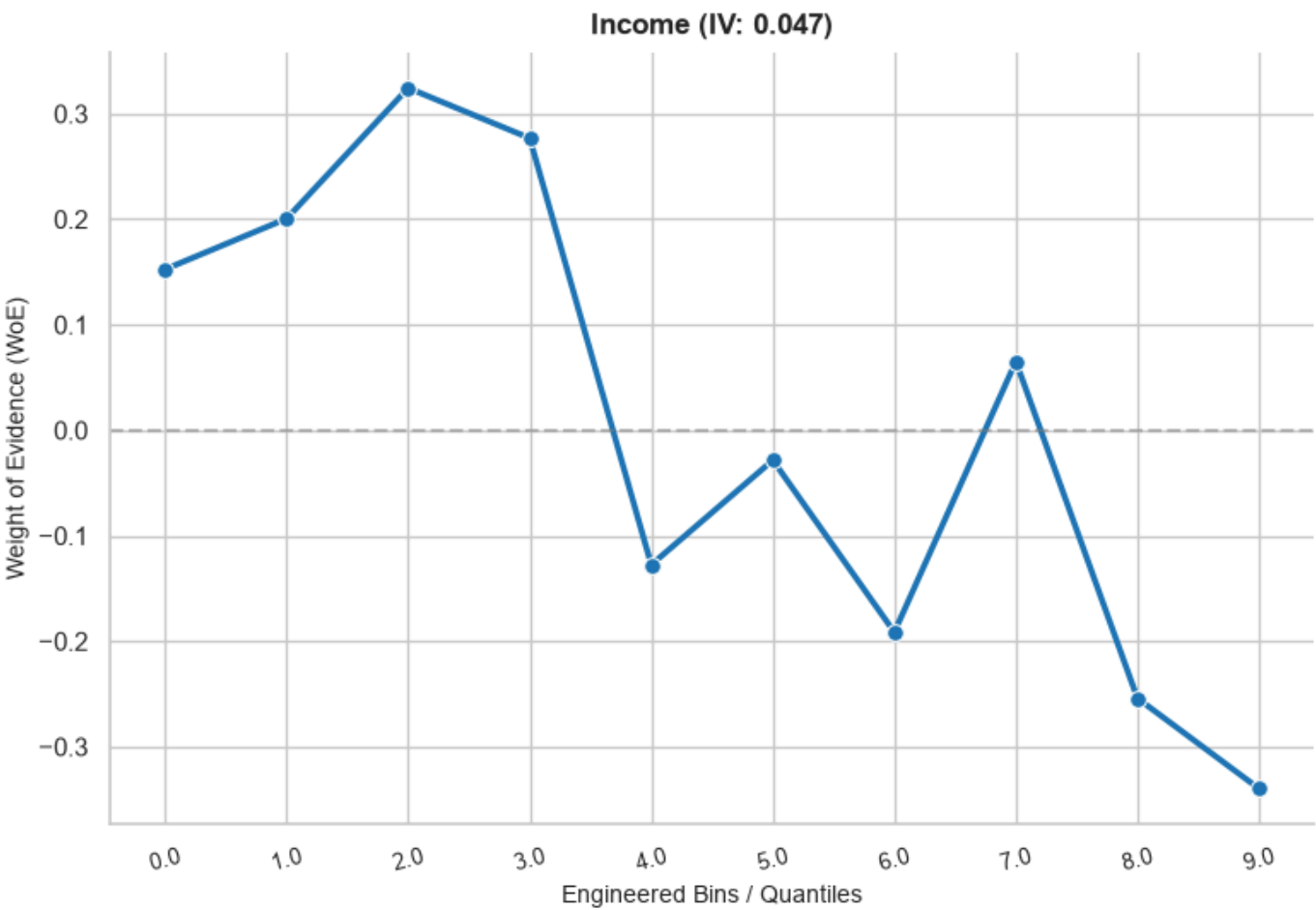
BackUp

Questions!!!

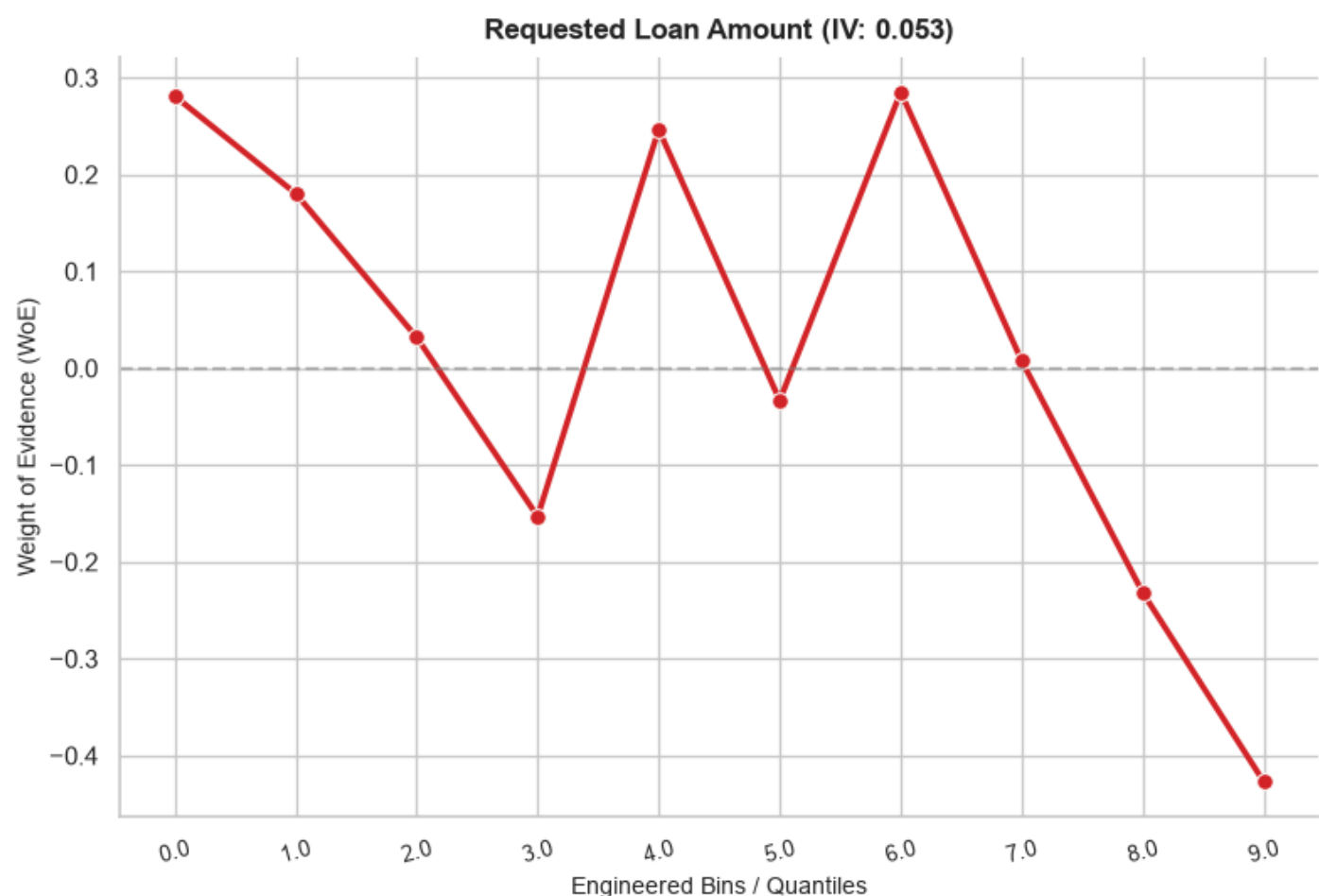
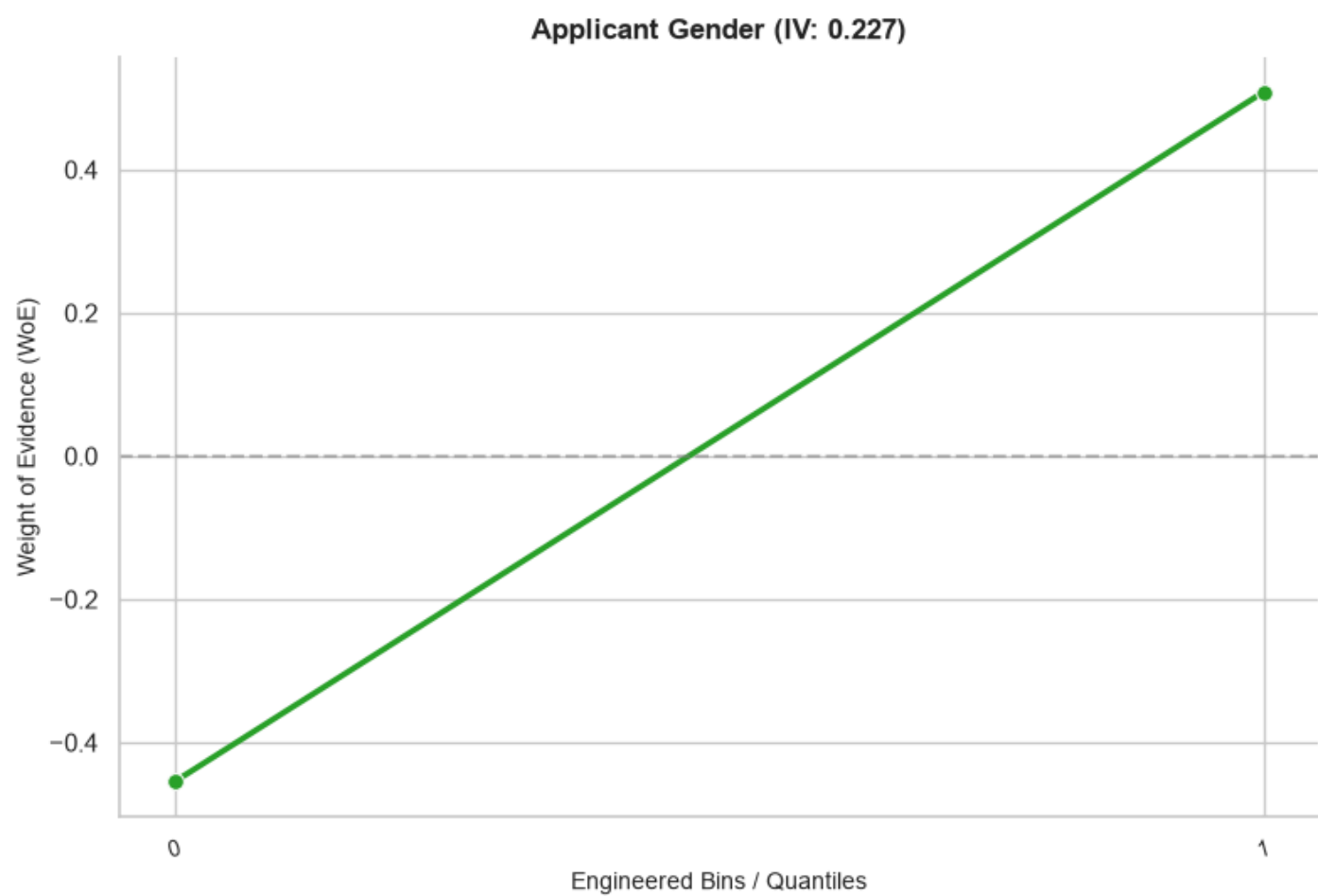
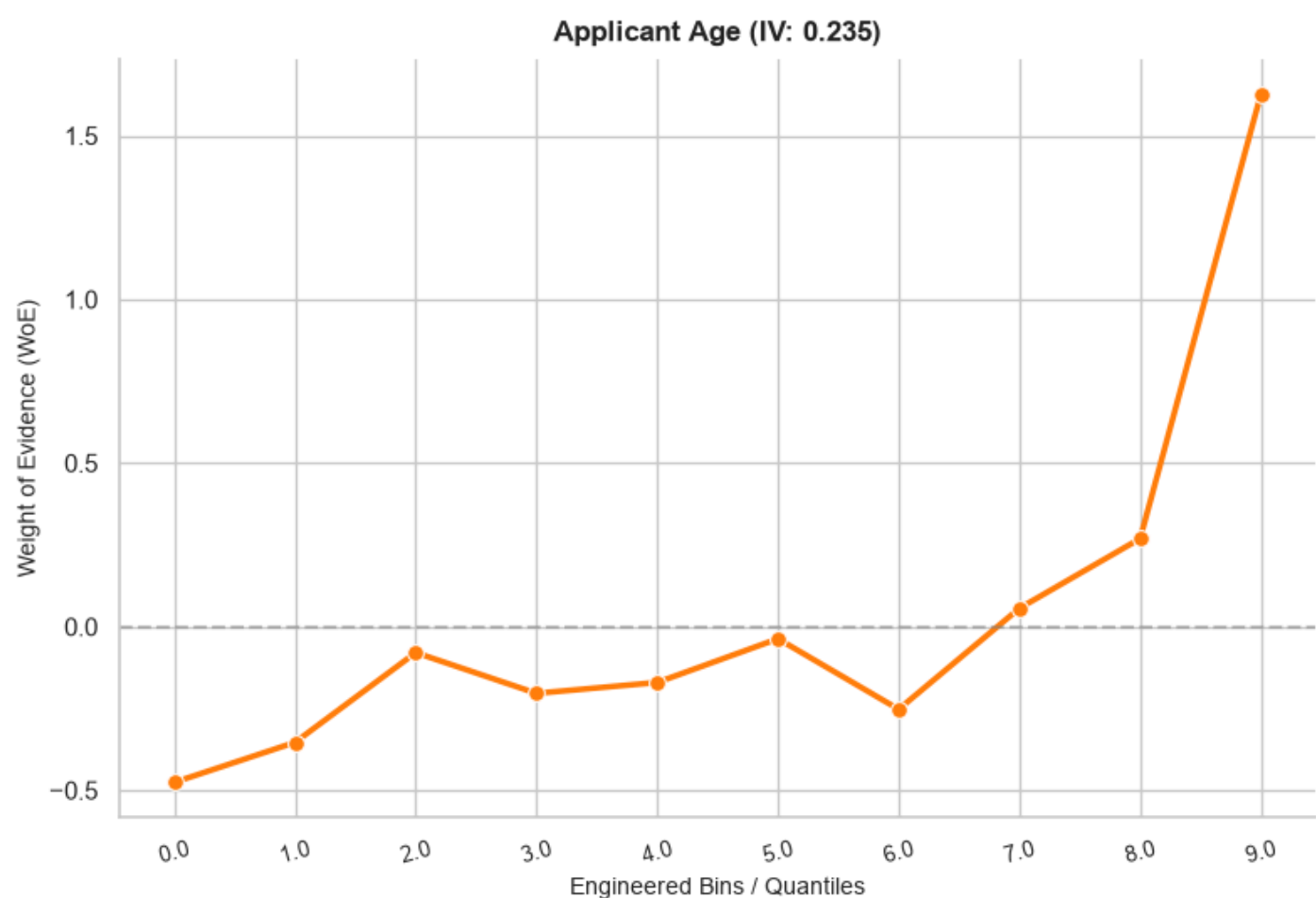
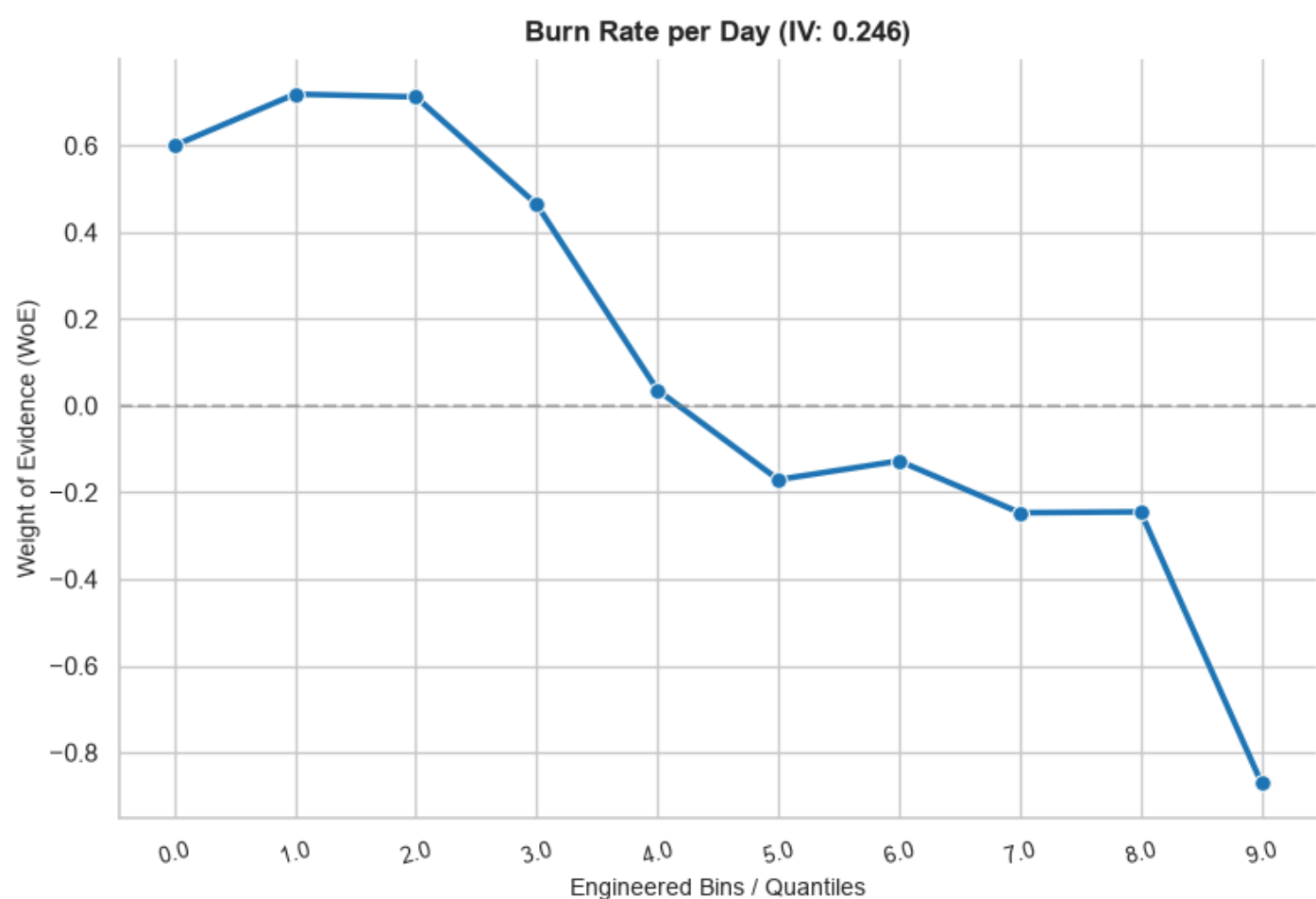
Pre-Modelling Feature Selection: Discriminant Power Ranking



WoE: Information Value <0.02



WoE: Information Value (0.1-0.5)



Grafana Dashboard

